



HiLCoE

School of Computer Science
& Technology

CarpoolConnect (Part I)

Members Alazar Gebremedhin
Delina Fikru
Henok Mesaye
Hiyab Yemane
Luliam Tekle
Remla Miftah

Course Object Oriented Software Engineering (CS446)

Batch/Section DRB2202/B

Instructor Axumawit Tassew

Term Spring, 2025

Date Aug 8, 2025

Contents

I Project Proposal	5
1 Background	5
2 Description of the Project	5
3 Statement of the Problem	5
4 Objectives	5
4.1 General Objectives	5
4.2 Specific Objectives	6
5 Scope	6
5.1 In Scope	6
5.2 Out of Scope	6
6 Methodology	7
7 Cost of Budget	7
8 Project Schedule	8
8.1 Work Break Down Structure	8
8.2 Project Schedule	8
9 Current System	10
II Proposed System	11
10 Function Definition	11
10.1 Functional Requirements	11
10.1.1 Authentication & Authorization	11
10.1.2 User & Profile Management	11
10.1.3 Driver Functionality	11
10.1.4 Passenger Functionality	12
10.1.5 Ride Matching & Community Features	12
10.1.6 Admin Panel Functionality	12
10.2 Non-functional Requirements	12
10.2.1 Performance	12
10.2.2 Reliability & Availability	13
10.2.3 Security	13
10.2.4 Scalability	13
10.2.5 Usability	13
10.2.6 Legal & Compliance	13
11 System Models	14
11.1 Use-case Models	14
11.1.1 Authentication and Profile Management Sub-System	14
11.1.2 Ride Lifecycle Management Sub-System	14
11.1.3 Community and Trust Sub-System	15
11.1.4 Administration and Support Platform Sub-System	15
11.2 Use-case Descriptions	16
11.2.1 Authenticate User via Social SSO	16
11.2.2 Deactivate User Account	18
11.2.3 Request a Ride	19
11.2.4 Track Active Ride	20
11.2.5 Cancel a Ride Request	21

11.2.6	Create and Offer a One-Time Ride	22
11.2.7	Manage Ride Requests	23
11.2.8	Start and Complete a Trip	24
11.2.9	Verify Driver Documents	25
11.2.10	Suspend a User Account	26
11.2.11	Investigate Ride Dispute	27
11.2.12	Respond to SOS Alert	28
11.2.13	Rate a Completed Ride	29
11.2.14	Configure System Parameters	30
11.2.15	Log Out	31
11.3	Class diagrams	32
11.3.1	High-Level Package Diagram	32
11.3.2	Authentication & Profile Management	33
11.3.3	Ride Lifecycle Management	34
11.3.4	Community & Trust	34
11.3.5	Administration and Support	35
11.3.6	Core Services	35
11.4	Sequence diagrams	36
11.4.1	Authenticate User via Social SSO	36
11.4.2	Deactivate User Account	37
11.4.3	Request a Ride	37
11.4.4	Track Active Ride	38
11.4.5	Cancel a Ride Request	38
11.4.6	Create and Offer Recurring Ride	39
11.4.7	Manage Ride Requests	40
11.4.8	Start and Complete a Trip	41
11.4.9	Verify Driver Documents	42
11.4.10	Suspend a User Account	43
11.4.11	Investigate Ride Dispute	43
11.4.12	Respond to SOS Alert	44
11.4.13	Rate a Completed Ride	44
11.4.14	Configure System Parameters	45
11.4.15	Log Out	45
12	User Interface Mock-ups	46
12.1	Onboarding & Common User Flows	46
12.2	Passenger User Flow	49
12.3	Driver User Flow	53
12.4	Admin Panel Flow (Web SPA)	57
	References	59

List of Figures

1	Work Breakdown Structure (WBS) for <i>CarpoolConnect</i> .	8
2	Project scheduling duration. Note that each regular sprint lasts 1 week and no work on Sunday's is assumed.	10
3	This diagram shows the fundamental interactions a user has with their account and session state.	14
4	This diagram outlines the core operational flow of the application, detailing how passengers request rides and how drivers offer and execute them.	14
5	This diagram illustrates the key processes for building a safe and reliable community through user ratings and administrative verification.	15
6	This diagram shows the tools available to administrators for managing the platform, its users, and its core business logic.	15
7	This diagram highlights the critical, high-priority workflow for handling in-app SOS alerts.	16
8	This diagram provides the architectural overview, showing how the different parts of the system interact.	32

9	This diagram focuses solely on users and their vehicles. It is the foundation for all other parts of the system.	33
10	This diagram details the core logic of creating and managing rides. Note the use of <<stub>> classes for User and DriverProfile to show the dependency without including their full details. Also please note that most of the aggregation / composition container attributes, e.g., <code>riders: List<Authentication::User></code> are simply implied because space constraint.	34
11	This diagram focuses on the entities that build safety and accountability.	34
12	This diagram shows the entities used by the internal admin team.	35
13	This diagram shows the Matching Engine component and its dependencies.	35
14	Sequence diagram showing SSO authentication flow.	36
15	Sequence diagram showing the deactivation of user account.	37
16	Sequence diagram showing requesting a ride.	37
17	Sequence diagram showing tracking of an active ride.	38
18	Sequence diagram showing the process of cancelling a ride.	38
19	Sequence diagram showing creating and offering recurring ride.	39
20	Sequence diagram showing managing of ride requests.	40
21	Sequence diagram showing the start and completion of a trip.	41
22	Sequence diagram showing the verification of driver documents.	42
23	Sequence diagram showing suspension of a user account.	43
24	Sequence diagram showing the investigation of ride disputes by Admin.	43
25	Sequence diagram showing response to SOS alert.	44
26	Sequence diagram showing the rating process of a completed ride.	44
27	Sequence diagram showing how super-user configures application parameters.	45
28	Sequence diagram showing the log our process.	45
29	Initial Login (UC-S-001). To provide a fast, password-free entry into the app.	46
30	Phone Number Verification (Post-SSO for New Users). To ensure every user has a verified, unique phone number before they can use any features.	47
31	OTP Entry (Post-Verification Code Sent). To complete the phone verification loop.	48
32	User Profile & Settings (UC-P-001, UC-S-002). To allow users to manage their account details and access critical actions like logout and deactivation.	49
33	Passenger Home (UC-PA-001). To provide the main entry point for requesting a ride.	50
34	Searching for Ride (Post-Request). To give feedback to the user that the system is actively working on their request.	51
35	Active Ride - Tracking (UC-PA-002). To provide real-time updates, driver details, and safety features during a ride.	52
36	Ride Rating (UC-G-001). To prompt the user to rate their driver after the trip is completed.	53
37	Driver Home - Default. To provide a clear starting point for a driver to offer a trip.	54
38	Offer Ride Form (UC-D-001/002). To allow the driver to specify all details for a new ride offer.	55
39	Pending Offer - Matched (UC-D-003). To show the driver their active offer and the passengers matched by the engine.	56
40	Active Trip - Navigation (UC-D-004). To provide turn-by-turn guidance for pickups and drop-offs.	57
41	Admin - Verification Queue (UC-A-001). To allow admins to efficiently process pending driver verifications.	58
42	Admin - Verification Detail View. To show all necessary information for an admin to approve or reject a driver.	58
43	Admin - SOS Alert Dashboard (UC-A-004). To provide an immediate, high-priority view of a live emergency.	59

List of Tables

1	CarpoolConnect Estimated Project Budget Breakdown (Revised)	7
2	CarpoolConnect SCRUM Project Plan	9

Part I

Project Proposal

1 Background

Ethiopia's rapid urbanization has brought prosperity but also severe transportation challenges. In Addis Ababa alone, population growth has outpaced road-network expansion, leading to chronic traffic congestion during peak hours. Commuters queue for public minibuses ("blue taxis") and share taxis for hours, only to find overcrowded vehicles and inconsistent schedules. The resultant idling vehicles contribute significantly to air pollution: Addis Ababa's annual PM2.5 levels exceed WHO guidelines by over 300%. Road safety risks are also elevated, as overcrowding in minibuses and informal ride-sharing practices compromise passenger security.

Against this backdrop, a reliable, scalable, and community-centric carpooling solution can deliver multiple social and environmental benefits: reduced private car use, lower emissions, less road congestion, and improved commuter experience.

2 Description of the Project

CarpoolConnect is a mobile and web platform that matches Drivers—everyday vehicle owners willing to share their cars—with Commuters seeking affordable, reliable rides along similar routes. Leveraging real-time GPS, route scheduling, and a credit-based reward system, CarpoolConnect transforms underutilized seat capacity into shared value. The solution encompasses:

- Driver App: for posting rides, setting availability, managing seats, and earning tokens.
- Commuter App: for discovering nearby rides, booking seats, subscribing to recurring trips, and safety reporting.
- Web Admin Portal: for moderation, dispute resolution, partner management, and analytics.
- Backend: for matching, token accounting, routing logic, notifications, and security.

3 Statement of the Problem

Urban commuters face significant daily challenges that impact their productivity and overall well-being. During peak rush hours, the average door-to-door journey frequently exceeds ninety minutes, leading to lost time and increased personal stress. This excessive travel duration is a direct result of severe traffic congestion, a problem that consistently undermines the efficiency of city transportation networks. These delays create a ripple effect, affecting work schedules, personal appointments, and the delicate balance between professional and private life for countless individuals.

The reliance on single-occupant vehicles creates substantial environmental and social costs for the community. Traffic emissions are a primary source of urban air pollution, contributing to a significant decline in air quality and posing long-term public health risks. At the same time, existing public transportation systems often fail to provide a reliable alternative. Issues such as overcrowded conditions and unpredictable schedules make public transit an uncomfortable and often impractical option for daily travel, forcing more people to depend on private cars and further worsening the core problem.

This situation reveals a critical inefficiency in how transportation assets are currently used. A large number of private cars travel with only one person, leaving multiple seats empty and available. This widespread underutilization of vehicle capacity represents a major missed opportunity to alleviate traffic pressure and reduce emissions. By failing to connect individuals who share similar routes, the current system perpetuates a cycle of congestion and pollution, negatively affecting the economic vitality and quality of life within the city.

4 Objectives

4.1 General Objectives

The primary objective of CarpoolConnect is to provide a reliable and efficient community-based transportation platform that reduces traffic congestion and travel times for urban commuters. By intelligently matching

volunteer drivers with passengers traveling along similar routes, the application aims to decrease the number of single-occupant vehicles on the road. This will lead to lower carbon emissions and create a more sustainable and interconnected urban environment for everyone.

4.2 Specific Objectives

1. To develop a mobile application for both iOS and Android that allows users to seamlessly register as either a passenger or a driver.
2. To implement a robust matching algorithm that efficiently groups passengers into pools and assigns them to the most suitable volunteer driver based on route, time, and vehicle capacity.
3. To create a secure and user-friendly interface that includes real-time GPS tracking of trips, in-app communication, and a two-way rating system to build community trust.
4. To design a credit-based incentive system that rewards drivers for offering rides, encouraging consistent and reliable participation on the platform.
5. To deploy a comprehensive admin panel that equips support staff with the tools to manage users, resolve disputes, and respond effectively to emergencies.
6. To reduce the average commuter's daily travel time by at least 20% within the first year of operation in a target metropolitan area.
7. To achieve a 15% reduction in the number of single-occupant car trips among active users by promoting shared rides for daily commutes.

5 Scope

5.1 In Scope

- Core Mobile Apps: Driver & Commuter features for signup, ride posting, matching, booking, and rating.
- Recurring & One-Time Rides: Support for flexible, scheduled, and on-demand routes.
- Seat Management: Vehicle profiles, capacity settings, and trip-level seat adjustments.
- Safety & Reporting: Panic button, post-ride report form, admin dashboard for incident management.
- Token System: Accrual, balance display, and voucher issuance.
- Basic Admin Portal: User management, report resolution workflow, and token partner management.
- Notifications: Push and in-app messaging for ride status and safety alerts.
- Geo-matching Engine: Real-time proximity and schedule overlap logic.

5.2 Out of Scope

- Advanced ML Features: Behavior-based preference learning and automated re-routing AI (deferred to Phase II).
- Integrated Insurance & Legal Services: Third-party insurance partnerships and liability management modules (for future release).
- Enterprise Fleet Management: Full-scale corporate shuttle integration.
- Hardware Integrations: Dashcam or in-vehicle telematics.
- Multi-language Support Beyond English/Amharic (deferred phase).

6 Methodology

The development of CarpoolConnect will be managed using the SCRUM methodology, as its principles are ideally suited to the fast-paced and uncertain environment of a startup. This agile framework allows us to prioritize a rapid time to market by focusing on delivering a Minimum Viable Product (MVP) in the initial development cycles. By concentrating on core features such as user registration, ride requests, and basic matching, we can quickly build a functional version of the application. This working MVP is not only the fastest way to enter the market, but it also serves as a powerful and tangible asset for demonstrating progress and potential to investors.

Furthermore, SCRUM provides the necessary flexibility to navigate the expected changes in requirements that all new ventures face. As a startup, our initial assumptions about user needs and market demands must be tested, not taken as facts. The short, iterative cycles, known as sprints, create a crucial feedback loop. We can release features, gather real user data, and then adapt our product backlog based on what we learn. This process is essential for systematically testing our product-market fit and ensuring that we are building a solution that people actually want to use, rather than committing extensive resources to unverified features.

Finally, the SCRUM framework fosters a collaborative environment and naturally mitigates project risk. Daily stand-up meetings and sprint retrospectives ensure continuous communication and alignment within the development team and with key stakeholders. By delivering the product in small, functional increments, we avoid the pitfalls of long development cycles where major issues can go undiscovered for months. If a feature does not resonate with users, the loss is confined to a single sprint, allowing the team to pivot quickly with minimal wasted effort. This makes the entire development process more transparent, responsive, and resilient.

7 Cost of Budget

The following table outlines the estimated project budget required to develop and launch the CarpoolConnect platform. This financial projection is based on a lean startup model, designed to reflect the operational strategy of using a dedicated team of non-salaried contributors. All financial figures presented are in United States Dollars (USD) and are categorized by key operational areas. The budget is broken down into distinct phases, from pre-launch preparations through the first two years of growth, to provide a clear view of anticipated expenditures over time.

Table 1: CarpoolConnect Estimated Project Budget Breakdown (Revised)

Cost Category	Pre-Launch (3m)	Launch - Y1	Growth - Y2	Total
<i>Infrastructure & Hosting</i>				
Small VPS Hosting (2-4GB)	\$60	\$240	\$480	\$780
DDoS Protection	\$75	\$300	\$600	\$975
<i>Third-Party Services & APIs</i>				
Mapping & Geocoding API	\$0	\$1,200	\$3,600	\$4,800
SMS Gateway (OTP)	\$50	\$500	\$1,500	\$2,050
App Store Developer Fees	\$125	\$125	\$125	\$375
<i>Development & Project Management Tools</i>				
GitHub Team Plan	\$36	\$144	\$144	\$324
Jira Standard Plan	\$68	\$270	\$270	\$608
Figma Professional Plan	\$36	\$144	\$144	\$324
<i>Marketing & User Acquisition</i>				
Launch Promotions	\$500	\$2,500	\$5,000	\$8,000
<i>Legal & Administrative</i>				
Registration & Compliance	\$1,000	\$500	\$500	\$2,000
Subtotal	\$1,950	\$5,923	\$12,363	\$20,236
Contingency (15%)	\$293	\$888	\$1,854	\$3,035
Grand Total	\$2,243	\$6,811	\$14,217	\$23,271¹

Note: Costs for VS Code and PlantUML are \$0 as they are free to use.

8 Project Schedule

8.1 Work Break Down Structure

Instead of decomposing the project into processes or activities, our WBS decomposes the final product into a hierarchy of tangible features and deliverables. This product-oriented approach is essential for agile development as it directly mirrors the structure of a product backlog and focuses the team on delivering value in increments. According to the NASA Work Breakdown Structure (WBS) Handbook[1], a product-oriented WBS is preferred because it organizes the project around the system's actual components and end items, rather than the work activities involved. This ensures that each element of the WBS represents a "potentially shippable increment," which is a core tenet of the SCRUM framework.

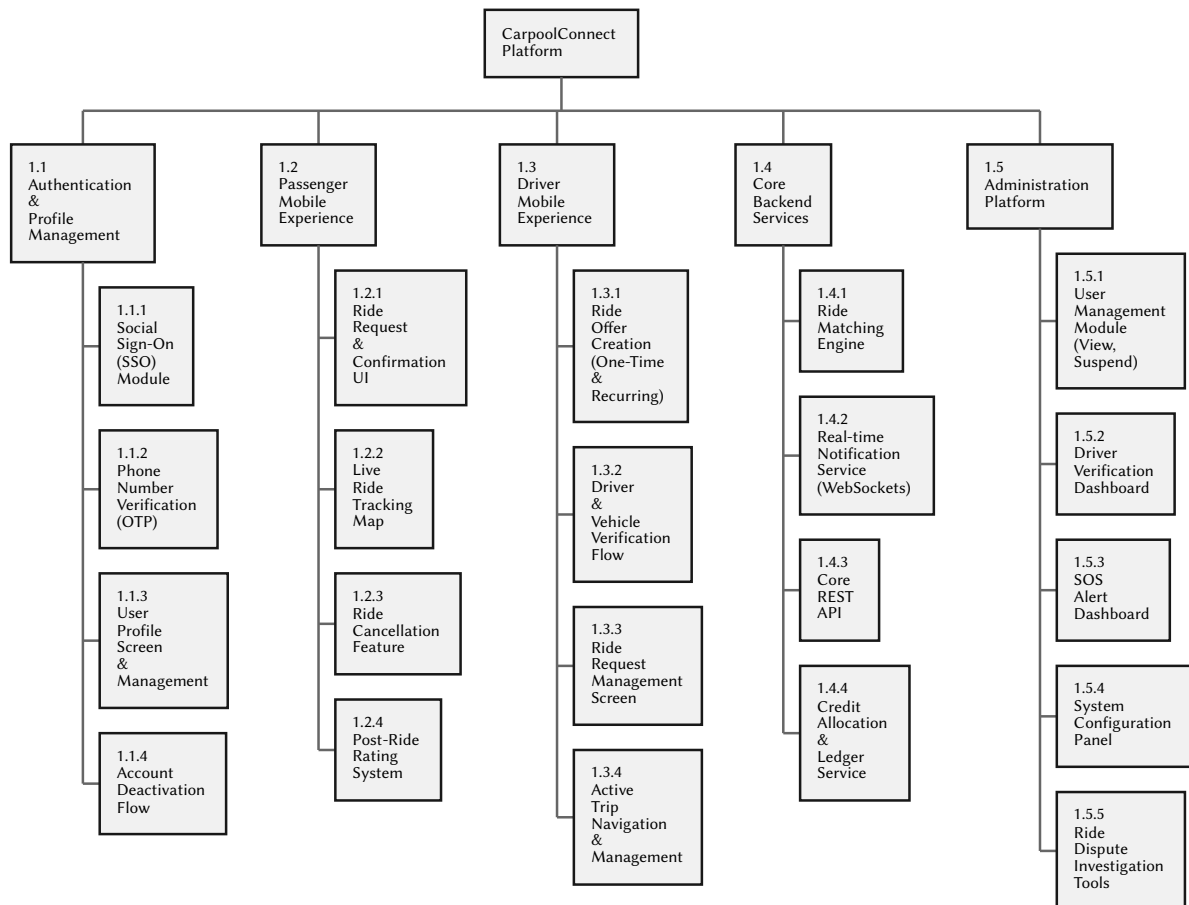


Figure 1: Work Breakdown Structure (WBS) for *CarpoolConnect*.

8.2 Project Schedule

The development plan for CarpoolConnect is structured around the SCRUM methodology to ensure an adaptive and incremental delivery process. The project will be organized into a series of one-week sprints, with each sprint focused on delivering a specific, functional piece of the application. The plan begins with a 1.5-week "Sprint 0" dedicated to project setup and product backlog grooming, ensuring all user stories are well-defined and prioritized before development starts. This is followed by a 15-week core development phase where the team will implement the use cases in a logical order, starting with foundational features and progressing through to administrative tools.

Phase	Timeline	Sprint(s)	Key Deliverables (Use Cases)	Primary Goal
Phase 1: Project Preparation				
Preparation & Grooming	1.5 Weeks	Sprint 0	Prioritized Product Backlog Development Environment Setup	Ensure team alignment and a well-defined plan to begin work.
Phase 2: Core Development (MVP Build)				
Module 1: Foundations	2 Weeks	Sprint 1 Sprint 2	UC-S-001: Authenticate via SSO UC-S-002: Log Out UC-P-001: Deactivate Account	Establish user identity and session management as the core platform foundation.
Module 2: Passenger Flow	3 Weeks	Sprint 3 Sprint 4 Sprint 5	UC-PA-001: Request a Ride UC-PA-003: Cancel a Ride Request UC-PA-002: Track Active Ride	Build the complete end-to-end experience for the demand side of the marketplace.
Module 3: Driver Flow	3 Weeks	Sprint 6 Sprint 7 Sprint 8	UC-D-001: Offer Recurring Ride UC-D-002: Offer One-Time Ride UC-D-003: Manage Ride Requests UC-D-004: Start and Complete Trip	Build the complete end-to-end experience for the supply side, enabling drivers to offer trips.
Module 4: Community & Trust	1 Week	Sprint 9	UC-G-001: Rate a Completed Ride	Implement the core feature for building trust and accountability.
Module 5: Admin Tools	5 Weeks	Sprint 10 Sprint 11 Sprint 12 Sprint 13 Sprint 14	UC-A-001: Verify Driver Documents UC-A-002: Suspend a User Account UC-A-003: Investigate Ride Dispute UC-A-005: Configure System Parameters UC-A-004: Respond to SOS Alert	Develop the essential backend tools needed for platform moderation, support, and safety.
Module 6: Hardening	1 Week	Sprint 15	Integration Testing & Bug Fixes	Stabilize the complete feature set and prepare for formal QA.
Phase 3: Testing & ReleasePreparation				
Final QA & Polish	2 Weeks	Post-Sprints	End-to-End System Testing App Store Submission Preparation	Ensure the MVP is stable, secure, and ready for a public launch.

Table 2: CarpoolConnect SCRUM Project Plan

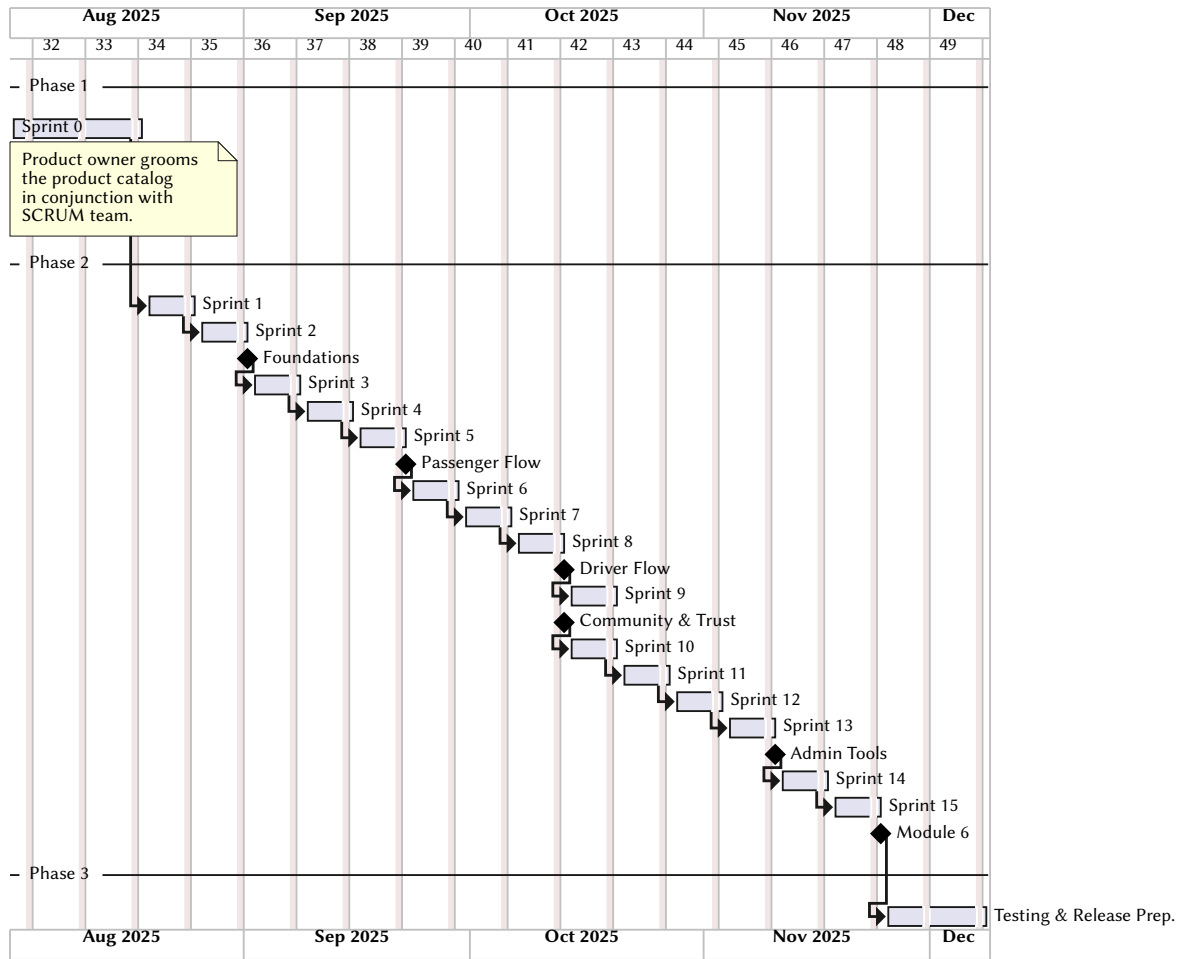


Figure 2: Project scheduling duration. Note that each regular sprint lasts 1 week and no work on Sunday's is assumed.

9 Current System

The existing transportation system in Ethiopia is a complex mix of traditional services and emerging modern solutions that do not fully address the needs of commuters. For decades, the primary modes of public transport have been the iconic blue and white minibus taxis and the large-capacity city buses, like those run by *Anbessa*. These services form the backbone of urban mobility, yet they are often characterized by fixed routes, unpredictable schedules, and significant overcrowding, particularly during peak hours. This legacy infrastructure struggles to cope with the demands of a rapidly growing urban population, leading to long wait times and general inconvenience for daily travelers.

In recent years, the landscape has been partially modernized by the arrival of ride-hailing applications like *Feres* and *RIDE*. These services have introduced a new level of convenience, offering on-demand private rides similar to the Uber model seen globally. While they have been widely adopted and have improved point-to-point travel, their operational model is still fundamentally based on providing individual taxi services. This means they contribute to, rather than solve, the underlying issues of traffic congestion and the high number of vehicles on the road, as they do not facilitate the sharing of rides among passengers heading in the same direction.

This situation reveals a clear and under-served niche in the market. Globally, dedicated carpooling platforms such as *BlaBlaCar*, and previously *Waze Carpool*, have proven successful by connecting drivers who have spare seats with passengers traveling along the same long-distance or commuter routes. These true carpooling services are designed to optimize vehicle occupancy, directly reducing traffic and emissions. However, these major international carpooling applications are not currently available in the Ethiopian market, leaving a significant opportunity for a platform designed specifically to fill this gap.

Part II

Proposed System

10 Function Definition

10.1 Functional Requirements

The following enumerates the functional requirements grouped by major feature areas.

10.1.1 Authentication & Authorization

- **FR-AUTH-01:** The system shall allow new and returning users to sign in using OAuth 2.0-based Social Logins (Google, Apple).
- **FR-AUTH-02:** The system shall automatically provision a new user account upon the first successful social sign-in.
- **FR-AUTH-03:** The system shall require all new users to verify their phone number via a One-Time Passcode (OTP) sent via SMS before accessing core app features.
- **FR-AUTH-04:** The system shall implement a secure session management mechanism using refresh and access tokens with a default session duration of 7 days.
- **FR-AUTH-05:** The system shall provide a secure method for users to log out, which must invalidate the session on the server side.

10.1.2 User & Profile Management

- **FR-UM-01:** The system shall allow users to view and update their profile information, including their name and profile picture.
- **FR-UM-02:** The system shall allow a user to switch between "Passenger" and "Driver" roles within the application, displaying the appropriate interface for the selected role.
- **FR-UM-03:** The system shall allow users to permanently deactivate their account. This action must be confirmable and render the user's PII inaccessible for active operations.
- **FR-UM-04:** The system shall maintain a history of all completed rides for each user, accessible from their profile.

10.1.3 Driver Functionality

- **FR-DRV-01:** The system shall allow verified drivers to offer a one-time ride by specifying a route, date, time, and number of available seats.
- **FR-DRV-02:** The system shall allow verified drivers to offer a recurring ride by specifying a route, a weekly schedule, a departure time, and available seats.
- **FR-DRV-03:** The system shall allow drivers to upload necessary verification documents (e.g., driver's license, vehicle registration).
- **FR-DRV-04:** The system shall notify drivers when passengers have been matched to their offered ride.
- **FR-DRV-05:** The system shall allow a driver to start a matched trip, which will update the ride status to "Active".
- **FR-DRV-06:** The system shall provide the driver with an optimized, turn-by-turn sequence of stops for all pickups and drop-offs.
- **FR-DRV-07:** The system shall allow a driver to complete a trip after the final passenger has been dropped off, which will trigger the credit allocation process.

10.1.4 Passenger Functionality

- **FR-PAX-01:** The system shall allow a passenger to request a ride by specifying a pickup location and a destination.
- **FR-PAX-02:** The system shall notify a passenger when a driver has been found and their ride is confirmed.
- **FR-PAX-03:** The system shall allow a passenger to track the real-time location of their assigned driver's vehicle on a map once the ride is active.
- **FR-PAX-04:** The system shall allow a passenger to cancel a requested or confirmed ride before the pickup time.

10.1.5 Ride Matching & Community Features

- **FR-MATCH-01:** The system's Matching Engine shall group passengers with similar routes and departure times into a "pool".
- **FR-MATCH-02:** The Matching Engine shall assign the most suitable driver to a pool based on route deviation, vehicle capacity, and driver schedule.
- **FR-RATE-01:** The system shall prompt both drivers and passengers to leave a rating (1-5 stars) and an optional review for each other after a ride is completed.
- **FR-SOS-01:** The system shall provide a prominent in-app SOS button for both drivers and passengers during an active ride.
- **FR-SOS-02:** Activating the SOS button shall immediately transmit the user's location, ride details, and identity to the emergency support dashboard.

10.1.6 Admin Panel Functionality

- **FR-ADMIN-01:** The system shall provide an admin panel for authorized staff to review and approve or reject driver verification documents.
- **FR-ADMIN-02:** The system shall allow admins to view detailed ride histories, including routes, timestamps, and communication logs, to investigate disputes.
- **FR-ADMIN-03:** The system shall allow authorized admins to suspend or deactivate user accounts for policy violations.
- **FR-ADMIN-04:** The system shall provide a real-time dashboard for the Emergency Response team to view and manage active SOS alerts.

10.2 Non-functional Requirements

The system shall satisfy the following quality attributes.

10.2.1 Performance

- **NFR-PERF-01:** API endpoints for core user actions (e.g., ride request, profile view) shall have a P95 response time of less than 400ms under normal load.
- **NFR-PERF-02:** The mobile application's home screen shall load completely within 3 seconds on a mid-tier smartphone with a stable 4G connection.
- **NFR-PERF-03:** Real-time location updates for a tracked vehicle shall appear on the passenger's map with a latency of no more than 2 seconds.
- **NFR-PERF-04:** The Matching Engine shall be able to process a new ride request and find a potential match within 60 seconds.

10.2.2 Reliability & Availability

- **NFR-REL-01:** All public-facing services (mobile API, web panel) shall achieve 99.5% uptime, calculated on a monthly basis and excluding scheduled maintenance windows.
- **NFR-REL-02:** The system shall ensure data integrity for all critical transactions (e.g., ride completion, credit allocation) through the use of ACID-compliant database operations.
- **NFR-REL-03:** In the event of a non-critical third-party service failure (e.g., mapping API for static maps), the application shall degrade gracefully without crashing.

10.2.3 Security

- **NFR-SEC-01:** All communication between the mobile client and the backend server shall be encrypted using TLS 1.2 or higher.
- **NFR-SEC-02:** All personally identifiable information (PII) and sensitive documents (e.g., driver's licenses) stored in the database shall be encrypted at rest.
- **NFR-SEC-03:** The system shall implement role-based access control (RBAC) for the admin panel to ensure staff can only access data and functions necessary for their role.
- **NFR-SEC-04:** The application shall be protected against the OWASP Top 10 common web application security risks.
- **NFR-SEC-05:** All user-uploaded documents must be stored in a secure, non-publicly accessible location and accessed via expiring, single-use URLs.

10.2.4 Scalability

- **NFR-SCAL-01:** The backend architecture shall be designed to scale horizontally to support an initial load of up to 10,000 concurrent active users.
- **NFR-SCAL-02:** The database system shall be capable of handling up to 1,000 write operations per minute during peak hours without significant performance degradation.
- **NFR-SCAL-03:** The Matching Engine shall be scalable independently of the main API services to handle increased computational load as the user base grows.

10.2.5 Usability

- **NFR-USE-01:** The mobile application shall provide a consistent user experience across both iOS and Android platforms.
- **NFR-USE-02:** The user interface shall be intuitive, requiring a new user no more than 5 minutes to learn how to request their first ride without assistance.
- **NFR-USE-03:** All user-facing error messages shall be clear, concise, and provide actionable guidance.

10.2.6 Legal & Compliance

- **NFR-LEGAL-01:** The system must obtain explicit user consent for collecting and processing personal data, including location data, in compliance with local data protection regulations.
- **NFR-LEGAL-02:** The system shall provide users with an accessible copy of the Terms of Service and Privacy Policy, which they must agree to during onboarding.
- **NFR-LEGAL-03:** All location data must be handled securely and used only for the purpose of providing the carpooling service as stated in the privacy policy.

11 System Models

11.1 Use-case Models

11.1.1 Authentication and Profile Management Sub-System

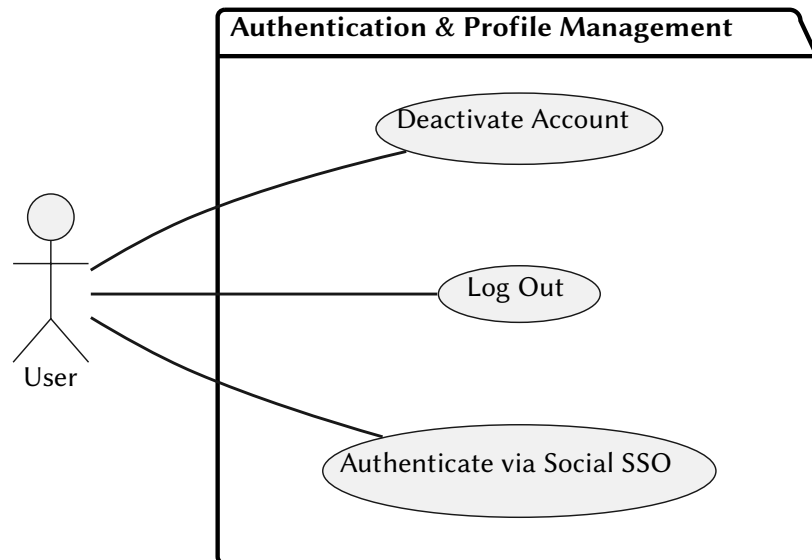


Figure 3: This diagram shows the fundamental interactions a user has with their account and session state.

11.1.2 Ride Lifecycle Management Sub-System

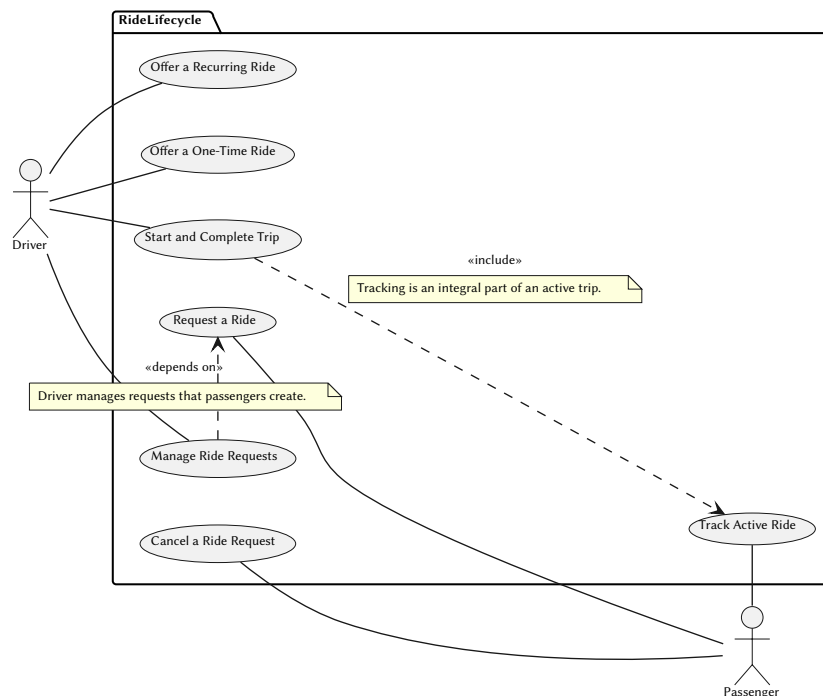


Figure 4: This diagram outlines the core operational flow of the application, detailing how passengers request rides and how drivers offer and execute them.

11.1.3 Community and Trust Sub-System

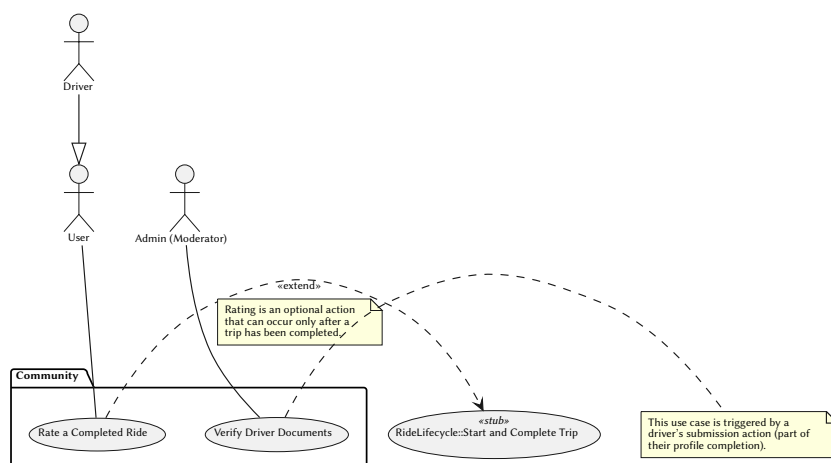


Figure 5: This diagram illustrates the key processes for building a safe and reliable community through user ratings and administrative verification.

11.1.4 Administration and Support Platform Sub-System

Due to its complexity, this platform is broken into two diagrams: “Platform Moderation & Operations” and “Emergency Response.”

Administration – Platform Moderation & Operations

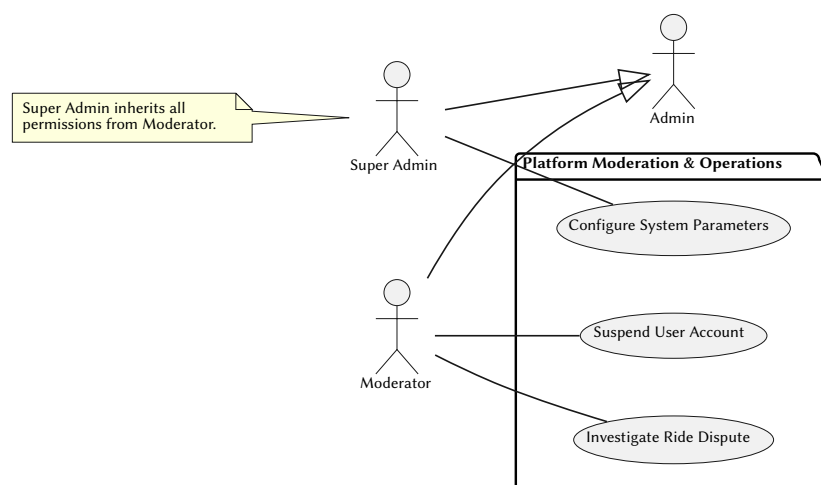


Figure 6: This diagram shows the tools available to administrators for managing the platform, its users, and its core business logic.

Emergency Response

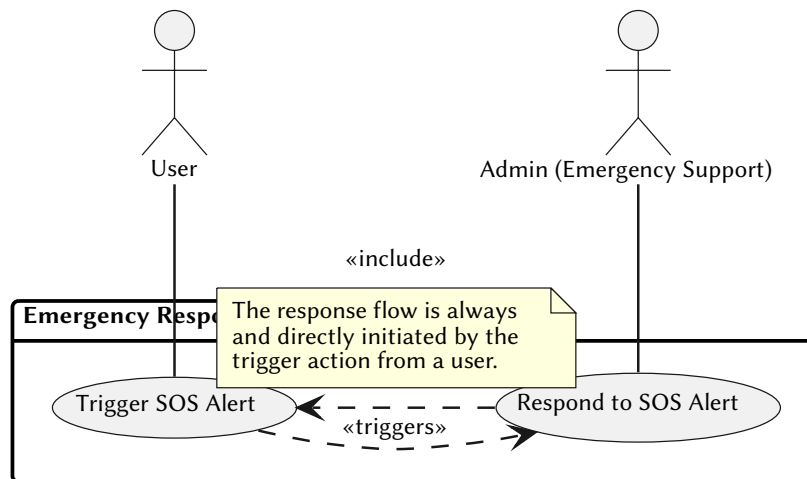


Figure 7: This diagram highlights the critical, high-priority workflow for handling in-app SOS alerts.

11.2 Use-case Descriptions

Note on notation:

- **S:** Stands for **Session**. These use cases relate to core system-level functions like authentication and session management.
- **P:** Stands for **Profile**. This prefix is used for use cases related to a user's personal profile management.
- **PA:** Stands for **Passenger**. This prefix identifies use cases initiated by or directly affecting the passenger persona.
- **D:** Stands for **Driver**. This prefix identifies use cases initiated by or directly affecting the driver persona.
- **G:** Stands for **General**. This prefix is used for universal use cases that apply to multiple actors, such as rating a ride.
- **A:** Stands for **Admin**. This prefix identifies use cases performed by an administrator within the backend admin panel.

11.2.1 Authenticate User via Social SSO

- **Description:** This use case describes how a new or returning user signs into the application using a third-party social media provider (e.g., Google, Apple). This simplifies login and registration by removing the need for a separate password.
- **Primary Actor:** Any User (Passenger or Driver)
- **Stakeholders and Interests:**
 - **User:** Wants a fast, secure, and easy way to access the app without remembering another password.
 - **System:** Wants to securely authenticate the user and retrieve basic profile information (name, email) to provision an account.
- **Entry conditions:**
 - The system is available.
 - The user has downloaded the CarpoolConnect app and is on the initial login screen.
 - The user has an active account with the chosen third-party SSO provider.
- **Exit conditions:**

- The user is successfully authenticated.
- A User record exists in the database (either newly created or retrieved).
- An active session is created for the user.
- If it's the user's first login, they are directed to the phone number verification screen.

- **Main Success Scenario (Basic Flow):**

1. User opens the app and is presented with login options.
2. User selects a social SSO provider (e.g., "Sign in with Google").
3. System redirects the user to the provider's authentication interface.
4. User enters their credentials and authorizes CarpoolConnect.
5. The SSO provider sends an authentication token back to the System.
6. System validates the token and retrieves user information.
7. System creates or updates the user's profile in the database and establishes a session.
8. Use case ends.

- **Extensions (Alternative Flows):**

- **4a. User cancels the SSO authorization:**
 - 4a1. The provider returns an error.
 - 4a2. System returns the user to the initial login screen.
 - 4a3. Use case ends.
- **6a. SSO token is invalid or expired:**
 - 6a1. System logs the error.
 - 6a2. System displays an error message: "Authentication failed. Please try again."
 - 6a3. System returns the user to the initial login screen.
 - 6a4. Use case ends.

- **Technology & Data Variations List:**

- **UI:** React Native mobile app.
- **Authentication:** OAuth 2.0 or OpenID Connect protocols.
- **Data:** User data (name, email, social_id) stored in the Users table in the SQL database.

- **Frequency of Occurrence:** High; occurs every time a user starts a new session.

- **Priority:** High

- **Scope:** CarpoolConnect Application

- **Level:** User Goal

- **Project-Specific Fields:**

- **ID:** UC-S-001
- **Version:** 1.0
- **Last Updated:** 2025-08-02

11.2.2 Deactivate User Account

- **Description:** This use case allows a user to permanently deactivate their account. This is an important privacy feature that removes their personal data from active use on the platform.
- **Primary Actor:** Any User (Passenger or Driver)
- **Stakeholders and Interests:**
 - **User:** Wants control over their data and the ability to leave the service permanently.
 - **System:** Needs to handle the deactivation gracefully, retaining anonymized data for historical reporting while disabling the user's access and removing personal information.
- **Entry conditions:**
 - The user is logged into their account.
 - The user has no active or pending rides.
- **Exit conditions:**
 - The user's account status is set to "Deactivated" in the database.
 - The user is logged out and their session is terminated.
 - Personal identifiable information (PII) is anonymized or deleted according to data retention policies.
- **Main Success Scenario (Basic Flow):**
 1. User navigates to their profile/settings screen.
 2. User selects the "Deactivate Account" option.
 3. System displays a confirmation prompt explaining the consequences.
 4. User confirms their intention to deactivate.
 5. System changes the AccountStatus in the Users table to "Deactivated".
 6. System terminates the user's session token.
 7. System logs the user out and returns them to the main login screen.
 8. Use case ends.
- **Extensions (Alternative Flows):**
 - **2a. User has an active ride:**
 - 2a1. System checks for active rides linked to the UserID.
 - 2a2. System displays a message: "You cannot deactivate your account while you have an active ride. Please complete or cancel your trip first."
 - 2a3. Use case ends.
- **Technology & Data Variations List:**
 - **Backend:** A REST API endpoint (e.g., POST /api/users/deactivate) triggers the deactivation logic.
 - **Data:** The Users table has an AccountStatus column. A background job may handle the data anonymization process.
- **Frequency of Occurrence:** Low
- **Priority:** Medium
- **Scope:** CarpoolConnect Application
- **Level:** User Goal
- **Project-Specific Fields:**
 - **ID:** UC-P-001
 - **Version:** 1.0
 - **Last Updated:** 2025-08-02

11.2.3 Request a Ride

- **Description:** A passenger requests transportation from a starting point to a destination for a specific time. This is the primary action for the passenger persona and initiates the matching process.
- **Primary Actor:** Passenger
- **Stakeholders and Interests:**
 - **Passenger:** Wants to find a convenient and affordable ride quickly.
 - **System:** Wants to capture the ride request and add it to the pool of requests to be processed by the matching algorithm.
 - **Driver:** Is interested in receiving notifications for passengers whose routes align with theirs.
- **Entry conditions:**
 - User is logged in and has a verified phone number.
 - User is in "Passenger" mode.
- **Exit conditions:**
 - A PoolRequest object is created and stored in the database with a "Requested" status.
 - The passenger's UI is updated to show that their request is being processed.
- **Main Success Scenario (Basic Flow):**
 1. Passenger opens the app, which displays a map centered on their current location.
 2. Passenger inputs their desired destination.
 3. Passenger confirms their pickup location (defaults to current location).
 4. Passenger confirms the ride details (route, estimated fare if applicable).
 5. Passenger submits the request.
 6. System creates a PoolRequest record in the database.
 7. System begins the matching algorithm to find a suitable driver or pool.
 8. System updates the passenger's screen to a "Searching for a driver..." state.
 9. Use case ends.
- **Extensions (Alternative Flows):**
 - **2a. Destination not found:**
 - 2a1. The mapping service cannot find the entered address.
 - 2a2. System prompts the user to enter a more specific address or drop a pin on the map.
 - 2a3. Use case returns to step 2.
- **Technology & Data Variations List:**
 - **UI:** React Native.
 - **Mapping:** Third-party map API for location search and display.
 - **Backend:** POST /api/pools/request endpoint to create the PoolRequest object in the SQL database.
- **Frequency of Occurrence:** High
- **Priority:** High
- **Scope:** CarpoolConnect Application
- **Level:** User Goal
- **Project-Specific Fields:**
 - **ID:** UC-PA-001
 - **Version:** 1.0
 - **Last Updated:** 2025-08-02

11.2.4 Track Active Ride

- **Description:** Once a passenger is matched with a driver, they can view the ride details and track the driver's vehicle in real-time on a map until they are dropped off.
- **Primary Actor:** Passenger
- **Stakeholders and Interests:**
 - **Passenger:** Wants to know the driver's ETA, see their current location, and feel secure about the trip's progress.
 - **Driver:** Wants the passenger to be ready for pickup and informed about their location.
 - **System:** Wants to provide a seamless and transparent in-trip experience.
- **Entry conditions:**
 - The passenger's PoolRequest has been matched and the Ride object status is "Matched" or "Active".
- **Exit conditions:**
 - The ride is completed or canceled.
- **Main Success Scenario (Basic Flow):**
 1. Passenger is on the active ride screen.
 2. System displays the driver's vehicle moving in real-time on the map.
 3. System displays the driver's details (name, photo, vehicle make, model, license plate).
 4. System displays the estimated time of arrival (ETA).
 5. Passenger uses the in-app chat to communicate with the driver if needed.
 6. System, via geofencing, automatically notifies the passenger when the driver is arriving.
 7. The ride proceeds until the passenger is dropped off.
 8. Use case ends.
- **Extensions (Alternative Flows):**
 - **2a. Connection is lost:**
 - 2a1. The WebSocket connection between the app and the server is interrupted.
 - 2a2. System displays the driver's last known location and a "Reconnecting..." status.
 - 2a3. System attempts to re-establish the connection.
- **Technology & Data Variations List:**
 - **Real-time:** WebSockets are used to push location data from the driver's device to the passenger's device.
 - **Geofencing:** The system uses geofences around pickup locations to trigger automated notifications.
 - **Data:** Ride object's currentLocation is continuously updated.
- **Frequency of Occurrence:** High
- **Priority:** High
- **Scope:** CarpoolConnect Application
- **Level:** User Goal
- **Project-Specific Fields:**
 - **ID:** UC-PA-002
 - **Version:** 1.0
 - **Last Updated:** 2025-08-02

11.2.5 Cancel a Ride Request

- **Description:** Allows a passenger to cancel their ride request before or after being matched with a driver.
- **Primary Actor:** Passenger
- **Stakeholders and Interests:**
 - **Passenger:** Wants the flexibility to change their plans.
 - **Driver:** Wants to be notified immediately if a passenger on their route cancels.
 - **System:** Needs to update the Ride status and potentially re-optimize the route for the driver if other passengers remain.
- **Entry conditions:**
 - The passenger has a PoolRequest with a status of "Requested" or "Matched".
- **Exit conditions:**
 - The passenger's request is removed from the pool.
 - The Ride object's status is updated to "Canceled" (or modified if other passengers remain).
 - The driver and passenger are notified.
- **Main Success Scenario (Basic Flow):**
 1. Passenger views their active ride request.
 2. Passenger selects the "Cancel Ride" option.
 3. System displays a confirmation dialog.
 4. Passenger confirms the cancellation.
 5. System updates the Ride status in the database.
 6. System notifies the assigned driver (if any) of the cancellation.
 7. System updates the passenger's UI to confirm the cancellation.
 8. Use case ends.
- **Technology & Data Variations List:**
 - **Backend:** POST /api/rides/{rideId}/cancel endpoint.
 - **Real-time:** WebSocket message sent to the driver's app for instant notification.
- **Frequency of Occurrence:** Medium
- **Priority:** High
- **Scope:** CarpoolConnect Application
- **Level:** User Goal
- **Project-Specific Fields:**
 - **ID:** UC-PA-003
 - **Version:** 1.0
 - **Last Updated:** 2025-08-02

11.2.6 Create and Offer a One-Time Ride

- **Description:** A verified driver offers their vehicle's spare space for a single, non-recurring trip (e.g., a weekend trip to another city).
- **Primary Actor:** Driver
- **Stakeholders and Interests:**
 - **Driver:** Wants to offset the cost of a trip they are already taking.
 - **Passenger:** Is interested in finding long-distance or non-standard routes.
 - **System:** Wants to increase the diversity and availability of offered rides.
- **Entry conditions:**
 - User is logged in, verified as a driver, and has a verified vehicle.
 - User is in "Driver" mode.
- **Exit conditions:**
 - A new Trip object is created in the database with a "Pending" status and a one-time schedule.
 - The trip is available to the matching engine.
- **Main Success Scenario (Basic Flow):**
 1. Driver selects "Offer a Ride".
 2. Driver chooses the "One-Time Ride" option.
 3. System presents a form.
 4. Driver enters the start location, destination, date, and departure time.
 5. Driver confirms the available seats.
 6. Driver submits the form.
 7. System validates the data and creates a Trip record.
 8. System confirms creation to the driver.
 9. Use case ends.
- **Technology & Data Variations List:**
 - **Backend:** POST /api/driver/trips endpoint.
 - **Data:** A Trip record is created in the database.
- **Frequency of Occurrence:** Medium
- **Priority:** High
- **Scope:** CarpoolConnect Application
- **Level:** User Goal
- **Project-Specific Fields:**
 - **ID:** UC-D-002
 - **Version:** 1.0
 - **Last Updated:** 2025-08-02

11.2.7 Manage Ride Requests

- **Description:** A driver reviews passengers who have been matched to their offered trip and accepts them to form a final "pool."
- **Primary Actor:** Driver
- **Stakeholders and Interests:**
 - **Driver:** Wants to see who they will be driving and confirm the final pool.
 - **Passenger:** Wants to receive a final confirmation that their ride is secured.
 - **System:** Wants to finalize the Ride object by getting driver confirmation.
- **Entry conditions:**
 - The driver has an offered Trip that has been matched with one or more PoolRequests by the system.
- **Exit conditions:**
 - The Ride object status is updated to "Matched".
 - The driver and all accepted passengers are notified.
 - An optimized route is generated for the trip.
- **Main Success Scenario (Basic Flow):**
 1. System notifies the driver of new passenger requests for their upcoming trip.
 2. Driver opens the trip details screen.
 3. System displays a list of matched passengers, including their rating and route segment.
 4. Driver reviews the passenger list.
 5. Driver clicks "Accept Pool" to confirm all matched passengers.
 6. System changes the Ride status to "Matched".
 7. System generates the OptimizedRoute and sends notifications to all parties.
 8. Use case ends.
- **Extensions (Alternative Flows):**
 - **5a. Driver rejects a passenger (if feature is enabled):**
 - 5a1. Driver chooses to remove a specific passenger from the suggested pool.
 - 5a2. System updates the pool and notifies the rejected passenger their request could not be fulfilled.
 - 5a3. Use case continues at step 5.
- **Technology & Data Variations List:**
 - **Backend:** POST /api/rides/{rideId}/accept endpoint.
 - **Algorithm:** The backend runs a route optimization algorithm after the pool is finalized.
- **Frequency of Occurrence:** High
- **Priority:** High
- **Scope:** CarpoolConnect Application
- **Level:** User Goal
- **Project-Specific Fields:**
 - **ID:** UC-D-003
 - **Version:** 1.0
 - **Last Updated:** 2025-08-02

11.2.8 Start and Complete a Trip

- **Description:** The driver initiates a matched trip, follows the optimized route for pickups and drop-offs, and marks the trip as complete.
- **Primary Actor:** Driver
- **Stakeholders and Interests:**
 - **Driver:** Wants clear, turn-by-turn directions and an easy way to manage the trip's lifecycle.
 - **Passenger:** Wants to be picked up and dropped off efficiently.
 - **System:** Wants to track the ride's progress in real-time and log its completion for credit allocation and auditing.
- **Entry conditions:**
 - The driver has a Ride with a "Matched" status.
 - The current time is at or near the scheduled departure time.
- **Exit conditions:**
 - The Ride object's status is moved from "Matched" -> "Active" -> "Completed".
 - All passengers are marked as dropped off.
 - The driver is awarded credits.
 - All parties are prompted to leave a rating.
- **Main Success Scenario (Basic Flow):**
 1. Driver opens the trip details and presses "Start Trip".
 2. System updates the Ride status to "Active".
 3. System displays the optimized route with the first pickup location.
 4. Driver drives to each pickup and drop-off location in the specified order.
 5. System uses geofencing to automatically detect and confirm each pickup and drop-off.
 6. After the final drop-off, the System prompts the driver to "Complete Trip".
 7. Driver confirms completion.
 8. System updates the Ride status to "Completed", allocates credits, and logs the trip.
 9. Use case ends.
- **Extensions (Alternative Flows):**
 - **5a. Geofence fails to trigger:**
 - 5a1. The driver has arrived but the system did not automatically detect it.
 - 5a2. System provides a manual override button (e.g., "Confirm Pickup") for the driver.
 - 5a3. Driver manually confirms the action, and the use case continues.
- **Technology & Data Variations List:**
 - **Real-time:** WebSockets for location broadcasting.
 - **Geofencing:** Automates status changes at pickup/drop-off points.
 - **Data:** The Ride object is the central entity, with its status and passenger pickup/dropoff statuses being updated throughout.
- **Frequency of Occurrence:** High
- **Priority:** High
- **Scope:** CarpoolConnect Application
- **Level:** User Goal
- **Project-Specific Fields:**
 - **ID:** UC-D-004
 - **Version:** 1.0
 - **Last Updated:** 2025-08-02

11.2.9 Verify Driver Documents

- **Description:** An admin reviews the documents (driver's license, vehicle registration) submitted by a new driver to ensure they meet the platform's safety and legal standards.
- **Primary Actor:** Admin (Support/Moderation Staff)
- **Stakeholders and Interests:**
 - **Admin:** Wants a clear and efficient interface to review documents and approve or reject applications.
 - **Driver:** Wants their application to be reviewed and approved quickly so they can start offering rides.
 - **System:** Wants to ensure that only verified drivers are active on the platform.
- **Entry conditions:**
 - Admin is logged into the Admin Panel.
 - A driver has submitted their documents for verification, placing them in a "Pending Verification" queue.
- **Exit conditions:**
 - The driver's `isVerified` status is set to `true` or `false`.
 - The driver is notified of the outcome.
- **Main Success Scenario (Basic Flow):**
 1. Admin navigates to the "Driver Verification" queue in the Admin Panel.
 2. Admin selects a driver from the queue.
 3. System displays the driver's profile information alongside images of their submitted documents.
 4. Admin reviews the documents for validity and accuracy.
 5. Admin clicks the "Approve" button.
 6. System updates the `User.isVerified` and `Vehicle.isVerified` flags to `true`.
 7. System sends an automated notification to the driver informing them of the approval.
 8. Use case ends.
- **Extensions (Alternative Flows):**
 - **5a. Documents are invalid or unclear:**
 - 5a1. Admin clicks the "Reject" button.
 - 5a2. System prompts the admin to select a reason for rejection (e.g., "Blurry Photo," "Expired Document").
 - 5a3. Admin selects a reason and submits.
 - 5a4. System updates the driver's status and sends a notification explaining the reason for rejection and inviting them to re-submit.
- **Technology & Data Variations List:**
 - **UI:** Web-based SPA (React, Vue, or Angular) for the Admin Panel.
 - **Data:** Interacts with the `Users` and `Vehicles` tables in the SQL database.
- **Frequency of Occurrence:** Medium (depends on new user growth)
- **Priority:** High
- **Scope:** CarpoolConnect Admin Panel
- **Level:** User Goal
- **Project-Specific Fields:**
 - **ID:** UC-A-001
 - **Version:** 1.0
 - **Last Updated:** 2025-08-02

11.2.10 Suspend a User Account

- **Description:** A Super Admin or Moderator suspends a user's account due to violations of community guidelines, safety issues, or fraudulent activity.
- **Primary Actor:** Admin (Super Admin or Moderator)
- **Stakeholders and Interests:**
 - **Admin:** Wants to quickly remove bad actors from the platform to maintain community safety and trust.
 - **System:** Wants to prevent a suspended user from accessing any platform features.
- **Entry conditions:**
 - Admin is logged into the Admin Panel with appropriate permissions.
 - A user has been identified for suspension based on reports or internal flags.
- **Exit conditions:**
 - The user's AccountStatus is set to "Suspended" in the database.
 - The user's active session is terminated.
 - A log of the action is created for auditing purposes.
- **Main Success Scenario (Basic Flow):**
 1. Admin searches for the user's profile in the Admin Panel.
 2. Admin navigates to the user's detail page.
 3. Admin clicks the "Suspend Account" button.
 4. System prompts the admin to enter a reason for the suspension.
 5. Admin enters the reason and confirms.
 6. System updates the User.AccountStatus to "Suspended" and logs the reason.
 7. System invalidates all of the user's active session tokens.
 8. Use case ends.
- **Technology & Data Variations List:**
 - **UI:** Web-based SPA.
 - **Data:** Action updates the AccountStatus in the Users table and creates a record in an AdminActionLog table.
- **Frequency of Occurrence:** Low
- **Priority:** High
- **Scope:** CarpoolConnect Admin Panel
- **Level:** User Goal
- **Project-Specific Fields:**
 - **ID:** UC-A-002
 - **Version:** 1.0
 - **Last Updated:** 2025-08-02

11.2.11 Investigate Ride Dispute

- **Description:** A Customer Support admin investigates a complaint filed by a user regarding a specific ride (e.g., a dispute over the route taken, a no-show, or a safety concern).
- **Primary Actor:** Admin (Customer Support)
- **Stakeholders and Interests:**
 - **Admin:** Wants access to all relevant data to make a fair and informed decision.
 - **User(s):** Want their issue to be heard and resolved fairly.
 - **System:** Wants to provide a complete and auditable record of every trip to facilitate dispute resolution.
- **Entry conditions:**
 - Admin is logged into the Admin Panel.
 - A support ticket has been created and linked to a specific `rideId`.
- **Exit conditions:**
 - The support ticket is updated with the investigation findings and resolution.
 - A record of the investigation is logged.
- **Main Success Scenario (Basic Flow):**
 1. Admin opens the support ticket from their queue.
 2. Admin uses the `rideId` to pull up the complete `Ride` object.
 3. System displays all associated data: passenger/driver details, planned vs. actual route map, all timestamps, geofence event logs, and the full `CommunicationLog` for the trip.
 4. Admin reviews all the evidence.
 5. Admin makes a decision (e.g., issue a credit, dismiss the complaint, send a warning).
 6. Admin documents the resolution in the support ticket system.
 7. Admin communicates the outcome to the user(s).
 8. Use case ends.
- **Technology & Data Variations List:**
 - **UI:** Web-based SPA with a "God's View" dashboard for ride history.
 - **Data:** This use case relies on the rich, interconnected data from the `Rides`, `Users`, `GeofenceEvents`, and `CommunicationLog` tables.
- **Frequency of Occurrence:** Medium
- **Priority:** High
- **Scope:** CarpoolConnect Admin Panel
- **Level:** User Goal
- **Project-Specific Fields:**
 - **ID:** UC-A-003
 - **Version:** 1.0
 - **Last Updated:** 2025-08-02

11.2.12 Respond to SOS Alert

- **Description:** An Emergency Support admin receives and acts upon an SOS alert triggered by a driver or passenger during an active ride. This is the highest priority support action.
- **Primary Actor:** Admin (Emergency Support)
- **Stakeholders and Interests:**
 - **Admin:** Wants to assess the situation and get help to the user as quickly as possible.
 - **User in Distress:** Wants immediate assistance.
 - **System:** Must provide accurate, real-time information instantly to the response team.
 - **Emergency Services:** May need accurate location data to dispatch help.
- **Entry conditions:**
 - A user has pressed the in-app SOS button.
 - An alert has been pushed to the Emergency Response dashboard.
- **Exit conditions:**
 - The situation is assessed and handled according to protocol.
 - The alert is logged and documented with all actions taken.
 - The incident is marked as resolved in the system.
- **Main Success Scenario (Basic Flow):**
 1. An audible and visual alarm appears on the Emergency Response dashboard.
 2. Admin immediately clicks on the alert.
 3. System displays the Ride details: live GPS location of the vehicle, user information for both driver and passengers, and their emergency contacts.
 4. Admin initiates a call to the user who triggered the alert via a dedicated line.
 5. Based on the user's response or lack thereof, the admin follows protocol, which may include calling the user's emergency contact or contacting local emergency services with the vehicle's precise location.
 6. Admin documents every action taken in the incident log.
 7. Once the situation is confirmed to be resolved, the admin closes the alert.
 8. Use case ends.
- **Extensions (Alternative Flows):**
 - **4a. User is unresponsive:**
 - 4a1. The admin cannot reach the user.
 - 4a2. Protocol dictates immediate escalation: contact the other party in the car (if applicable), then the emergency contact, then emergency services.
- **Technology & Data Variations List:**
 - **Real-time:** WebSockets are critical for the instant delivery of the SOS alert and live GPS data.
 - **UI:** A dedicated, high-priority dashboard in the Admin Panel that may be separate from the standard view.
- **Frequency of Occurrence:** Low (but critical)
- **Priority:** Critical
- **Scope:** CarpoolConnect Admin Panel
- **Level:** User Goal
- **Project-Specific Fields:**
 - **ID:** UC-A-004
 - **Version:** 1.0
 - **Last Updated:** 2025-08-02

11.2.13 Rate a Completed Ride

- **Description:** After a trip is completed, both the driver and passenger are prompted to leave a rating and review for each other to build community trust.
- **Primary Actor:** Passenger or Driver
- **Stakeholders and Interests:**
 - **User (Rater):** Wants to share their experience and contribute to the community's safety and quality.
 - **User (Rated):** Is interested in their own rating, which affects their ability to get rides/passengers.
 - **System:** Wants to collect rating data to build user profiles and identify high and low-performing users.
- **Entry conditions:**
 - A Ride has a status of "Completed".
- **Exit conditions:**
 - A Rating record is created in the database, linked to the rideId and both users.
 - The rated user's average rating is updated.
- **Main Success Scenario (Basic Flow):**
 1. After the ride is completed, the System sends a push notification to the user prompting them to rate their experience.
 2. User opens the rating screen.
 3. User selects a star rating (1-5).
 4. User (optionally) leaves a short text review.
 5. User submits the rating.
 6. System creates a Rating record in the database.
 7. System recalculates and updates the average rating for the user who was rated.
 8. Use case ends.
- **Technology & Data Variations List:**
 - **Data:** A Ratings table with columns for ratingId, rideId, raterId, ratedId, score, reviewText.
- **Frequency of Occurrence:** High
- **Priority:** High
- **Scope:** CarpoolConnect Application
- **Level:** User Goal
- **Project-Specific Fields:**
 - **ID:** UC-G-001 (General)
 - **Version:** 1.0
 - **Last Updated:** 2025-08-02

11.2.14 Configure System Parameters

- **Description:** A Super Admin adjusts core business logic parameters, such as the credit multiplier for recurring rides, without needing to deploy new code.
- **Primary Actor:** Super Admin
- **Stakeholders and Interests:**
 - **Super Admin:** Wants the flexibility to tune the platform's incentives and logic to respond to business needs.
 - **System:** Wants to use these configurable parameters in its core algorithms (matching, credit allocation).
- **Entry conditions:**
 - Admin is logged into the Admin Panel with Super Admin privileges.
- **Exit conditions:**
 - The configuration value is updated in the database.
 - The system's backend immediately begins using the new value for all subsequent calculations.
- **Main Success Scenario (Basic Flow):**
 1. Super Admin navigates to the "System Settings" or "Tunable Settings" section of the Admin Panel.
 2. System displays a form with various configuration fields (e.g., "Recurrent Ride Multiplier").
 3. Super Admin changes the value in one of the fields (e.g., from 2 to 2.5).
 4. Super Admin clicks "Save Changes".
 5. System validates the new value (e.g., must be a number).
 6. System updates the value in a Configuration table in the database.
 7. Use case ends.
- **Technology & Data Variations List:**
 - **UI:** Web-based SPA.
 - **Data:** A dedicated Configuration or Settings table in the database to store key-value pairs.
- **Frequency of Occurrence:** Low
- **Priority:** Medium
- **Scope:** CarpoolConnect Admin Panel
- **Level:** User Goal
- **Project-Specific Fields:**
 - **ID:** UC-A-005
 - **Version:** 1.0
 - **Last Updated:** 2025-08-02

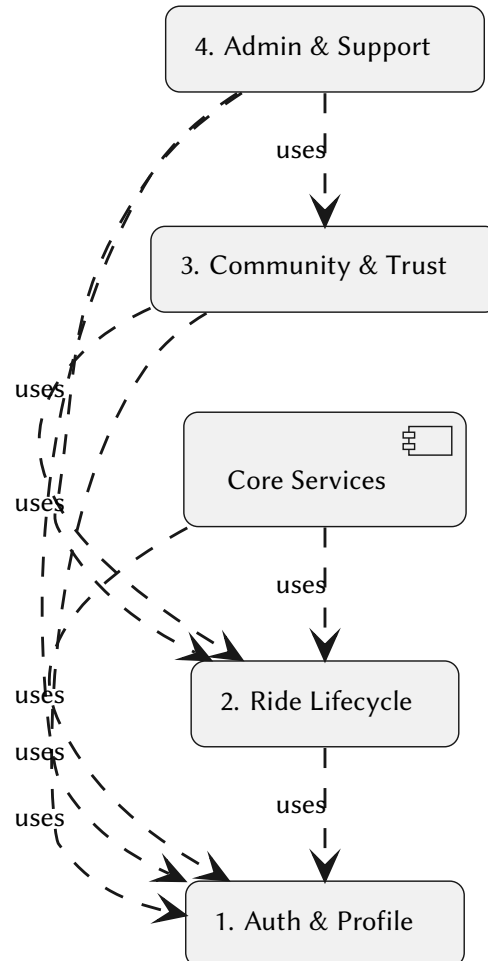
11.2.15 Log Out

- **Description:** An authenticated user securely ends their session with the application.
- **Primary Actor:** Any User
- **Stakeholders and Interests:**
 - **User:** Wants to ensure their session is terminated and no one else can use the app on their device under their account.
 - **System:** Wants to invalidate the user's session token to maintain security.
- **Entry conditions:**
 - The user is currently logged in.
- **Exit conditions:**
 - The user's session token is invalidated on the server.
 - The user is returned to the main login screen.
- **Main Success Scenario (Basic Flow):**
 1. User navigates to their profile or a menu.
 2. User selects the "Log Out" option.
 3. System displays a confirmation prompt.
 4. User confirms.
 5. System sends a request to the backend to invalidate the session token.
 6. System clears any local session data on the device.
 7. System navigates the user to the login screen.
 8. Use case ends.
- **Technology & Data Variations List:**
 - **Backend:** POST /api/auth/logout endpoint.
 - **Security:** Session tokens (e.g., JWT) may be added to a denylist/blocklist until they expire.
- **Frequency of Occurrence:** Medium
- **Priority:** High
- **Scope:** CarpoolConnect Application
- **Level:** User Goal
- **Project-Specific Fields:**
 - **ID:** UC-S-002
 - **Version:** 1.0
 - **Last Updated:** 2025-08-02

11.3 Class diagrams

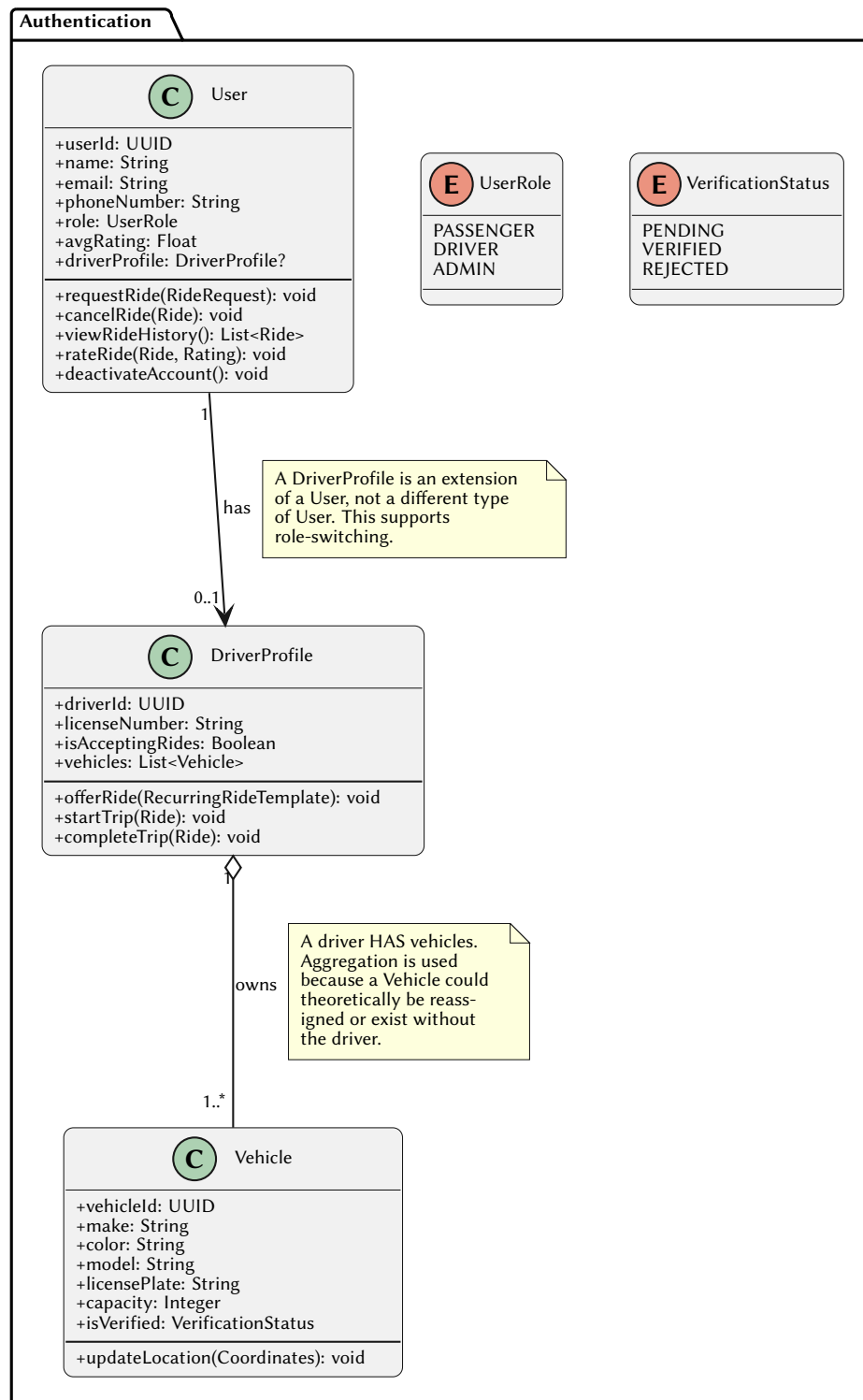
11.3.1 High-Level Package Diagram

Figure 8: This diagram provides the architectural overview, showing how the different parts of the system interact.



11.3.2 Authentication & Profile Management

Figure 9: This diagram focuses solely on users and their vehicles. It is the foundation for all other parts of the system.



11.3.3 Ride Lifecycle Management

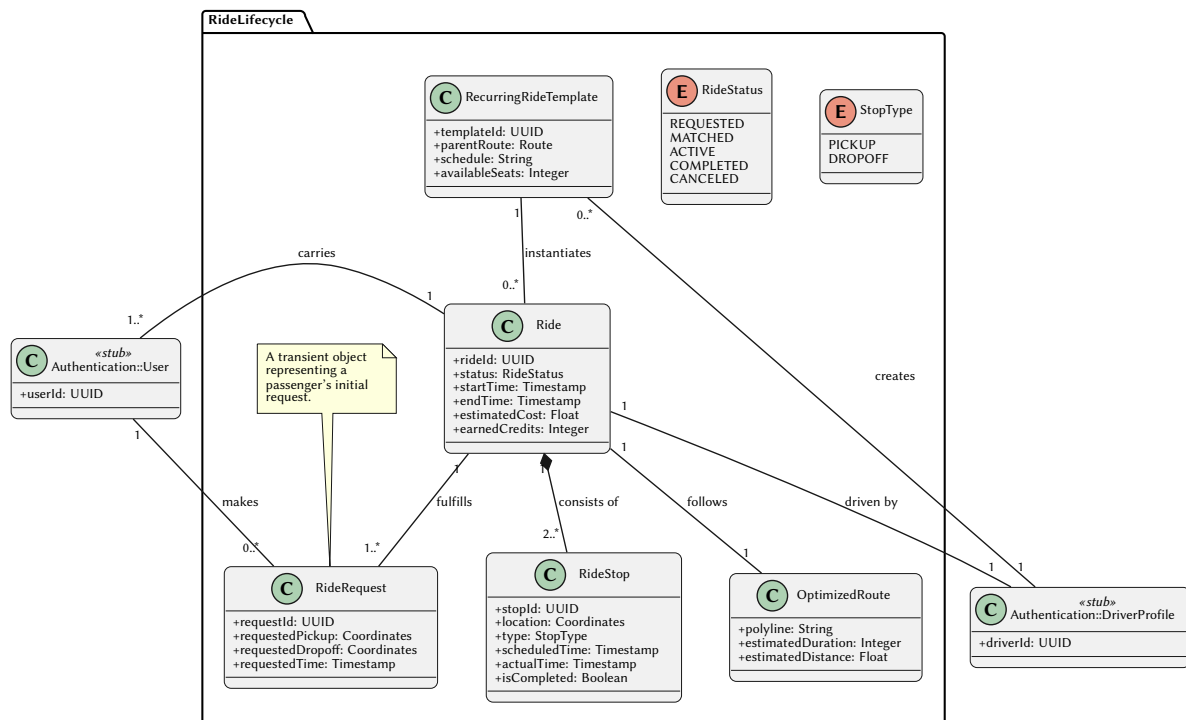
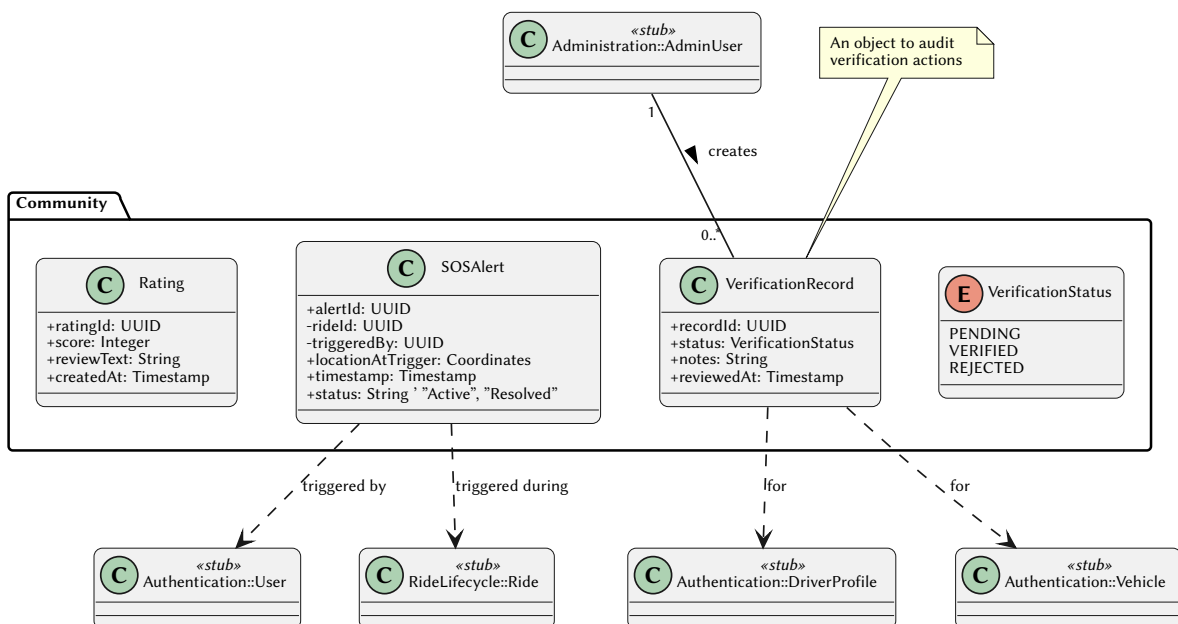


Figure 10: This diagram details the core logic of creating and managing rides. Note the use of `<<stub>>` classes for User and DriverProfile to show the dependency without including their full details. Also please note that most of the aggregation / composition container attributes, e.g., `riders: List<Authentication::User>` are simply implied because space constraint.

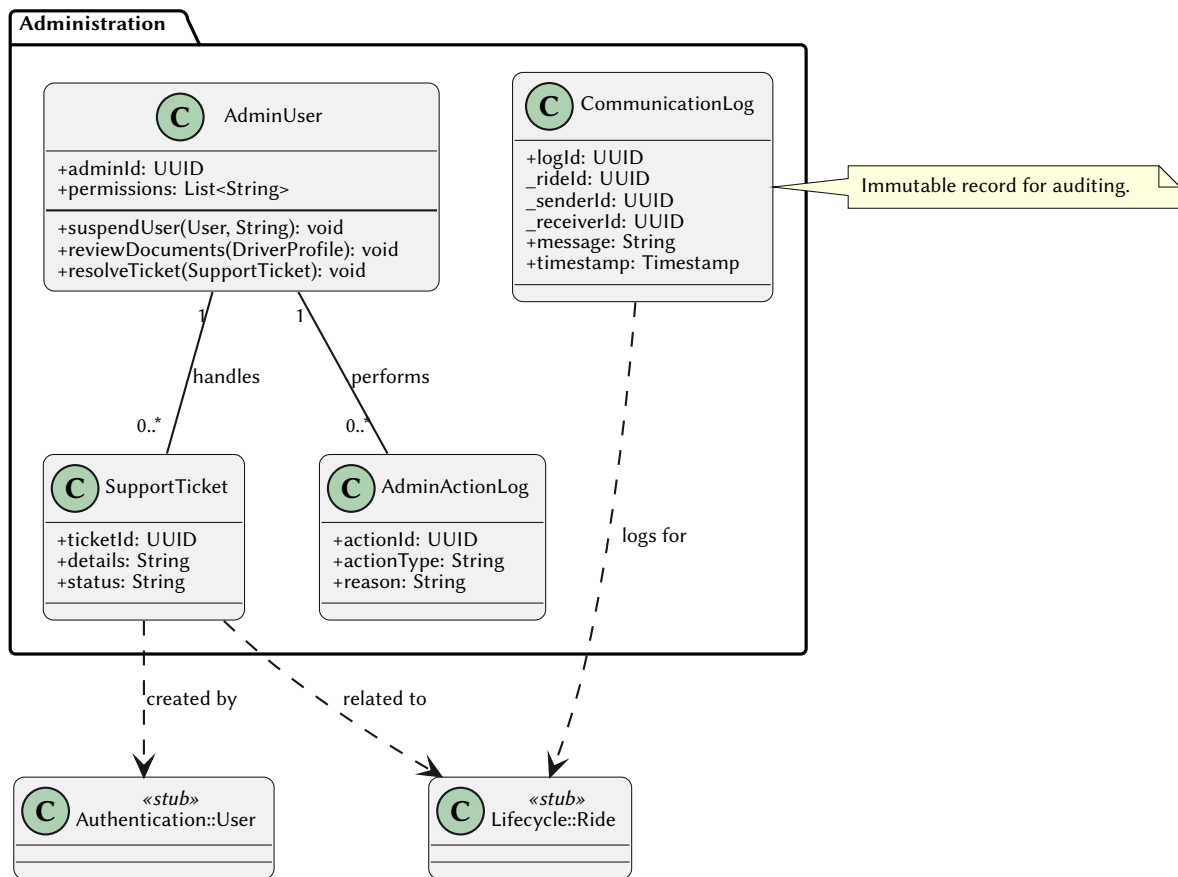
11.3.4 Community & Trust

Figure 11: This diagram focuses on the entities that build safety and accountability.



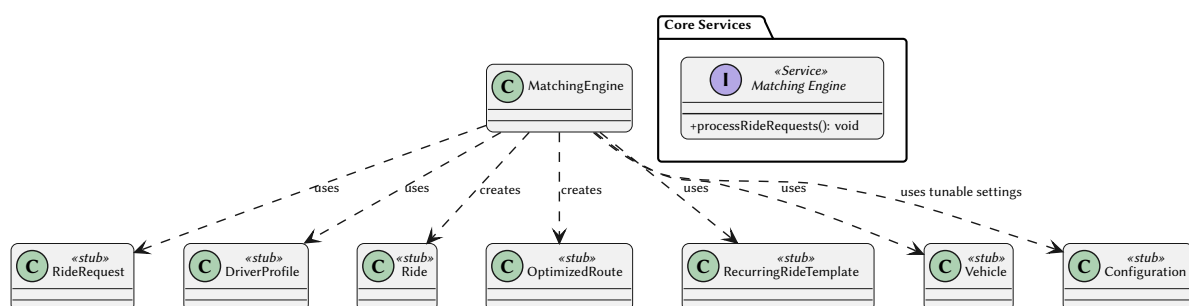
11.3.5 Administration and Support

Figure 12: This diagram shows the entities used by the internal admin team.



11.3.6 Core Services

Figure 13: This diagram shows the Matching Engine component and its dependencies.



11.4 Sequence diagrams

11.4.1 Authenticate User via Social SSO

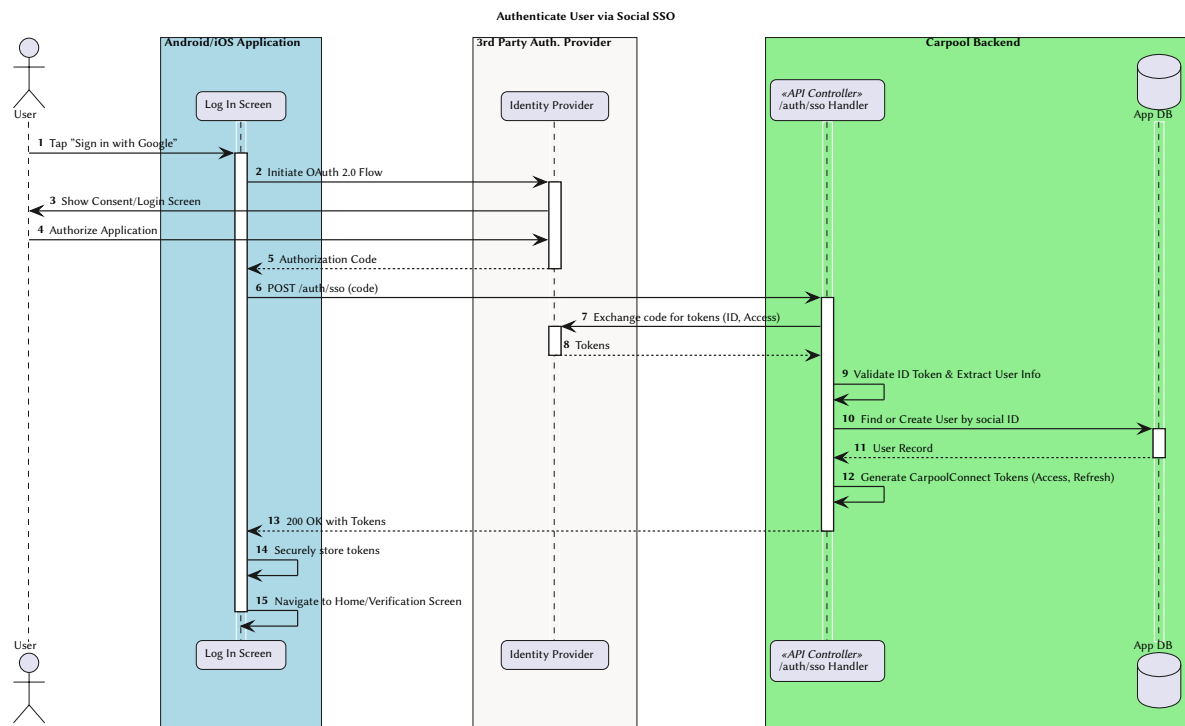


Figure 14: Sequence diagram showing SSO authentication flow.

11.4.2 Deactivate User Account

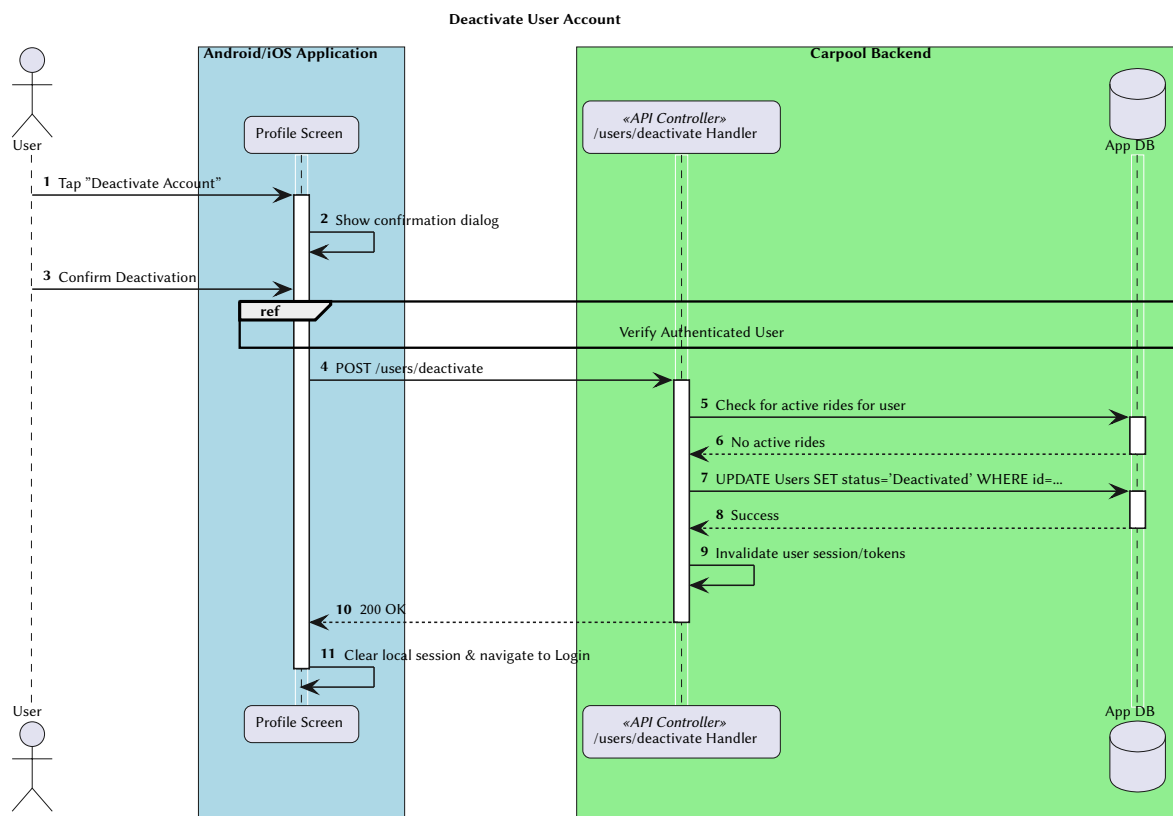


Figure 15: Sequence diagram showing the deactivation of user account.

11.4.3 Request a Ride

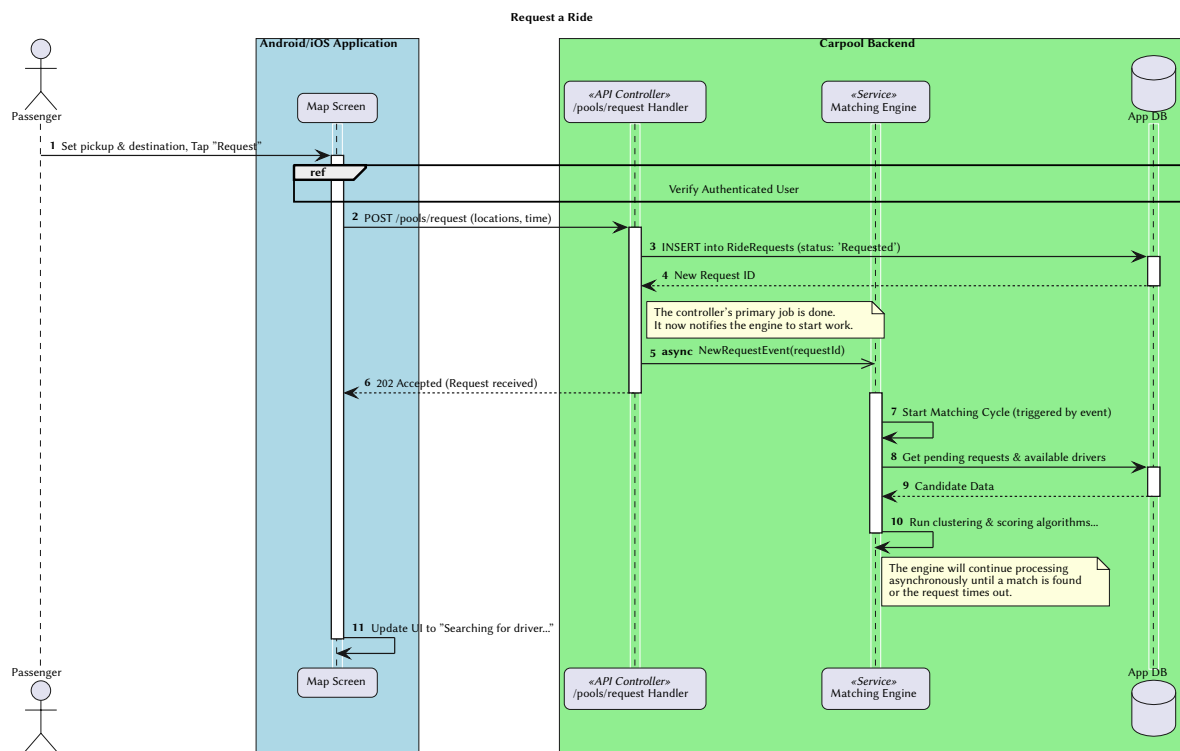


Figure 16: Sequence diagram showing requesting a ride.

11.4.4 Track Active Ride

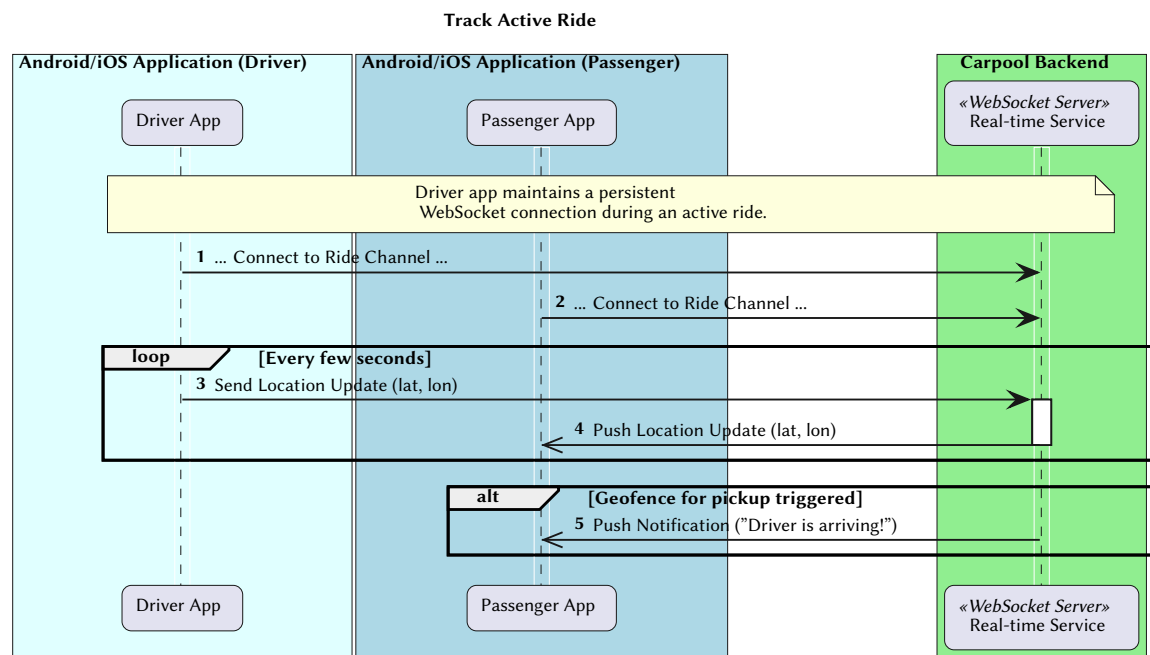


Figure 17: Sequence diagram showing tracking of an active ride.

11.4.5 Cancel a Ride Request

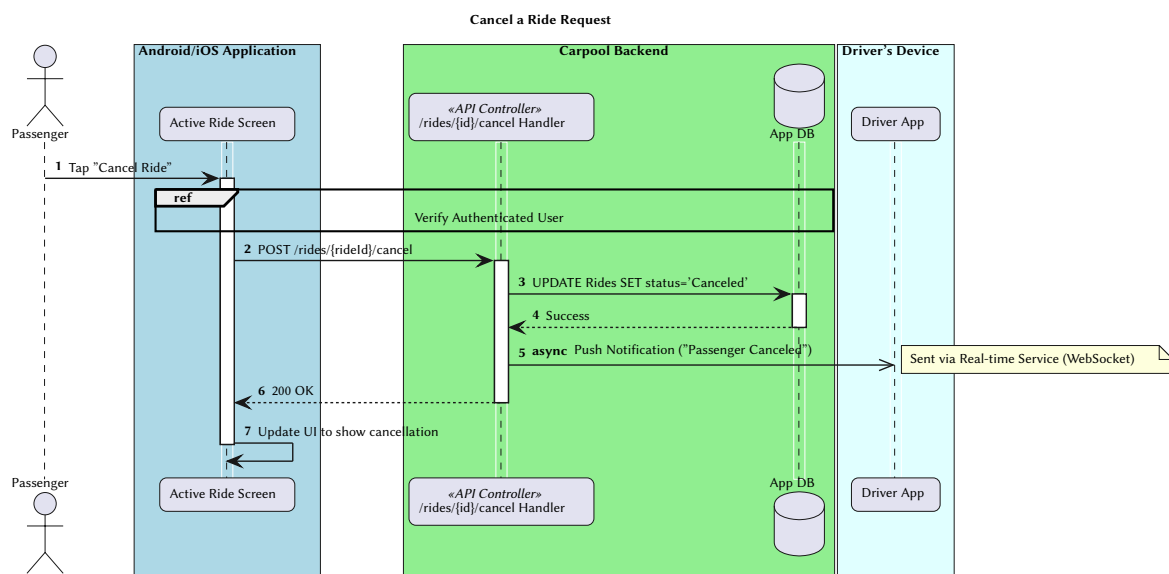


Figure 18: Sequence diagram showing the process of cancelling a ride.

11.4.6 Create and Offer Recurring Ride

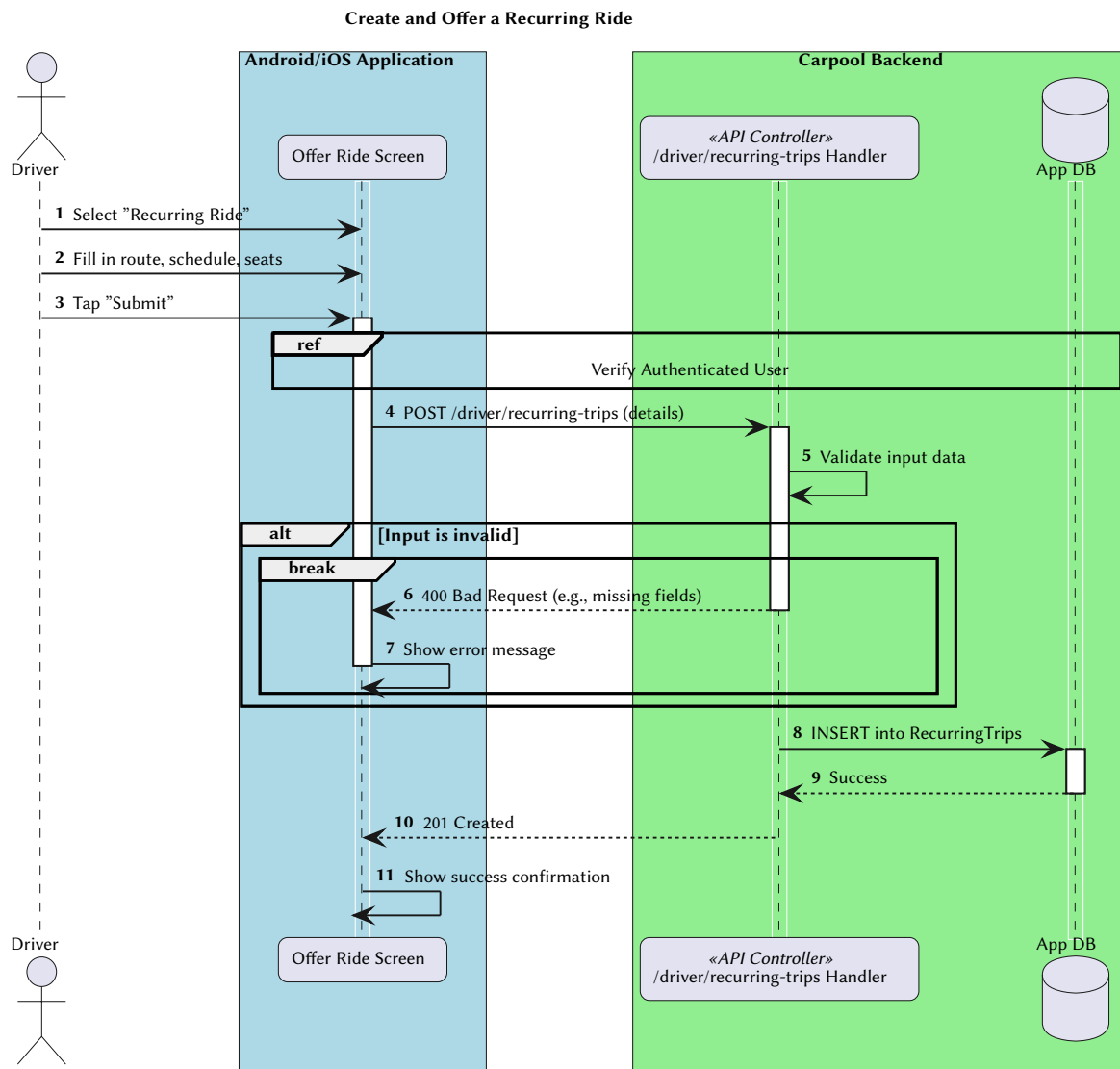


Figure 19: Sequence diagram showing creating and offering recurring ride.

11.4.7 Manage Ride Requests

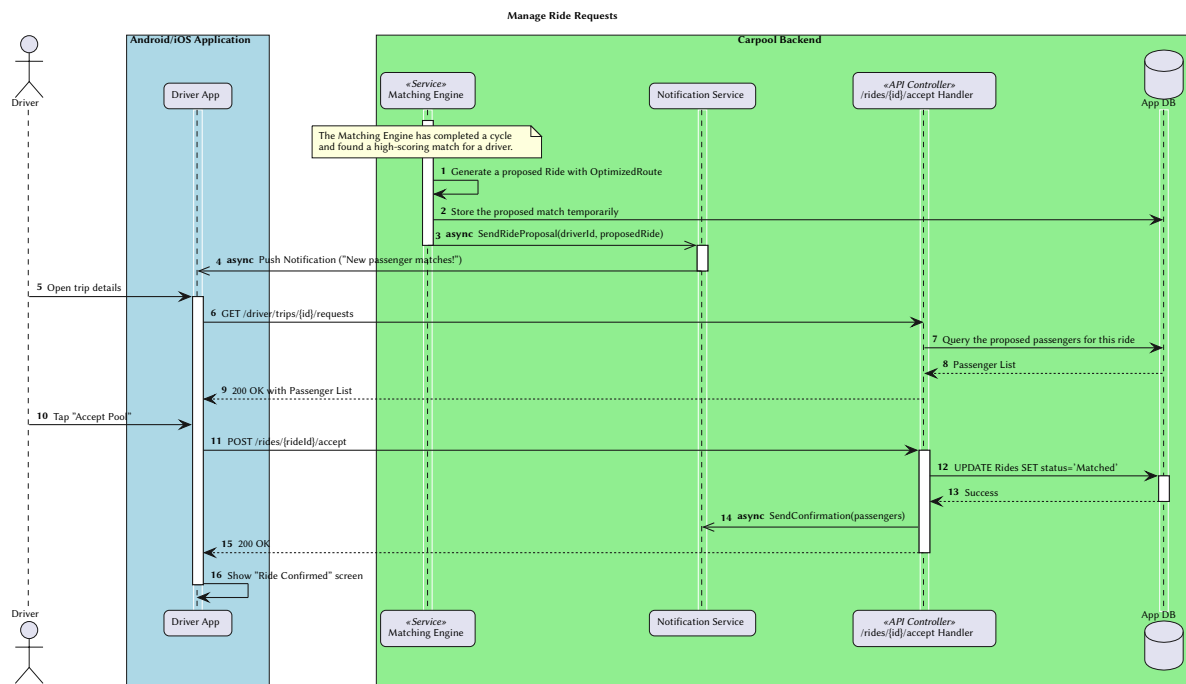


Figure 20: Sequence diagram showing managing of ride requests.

11.4.8 Start and Complete a Trip

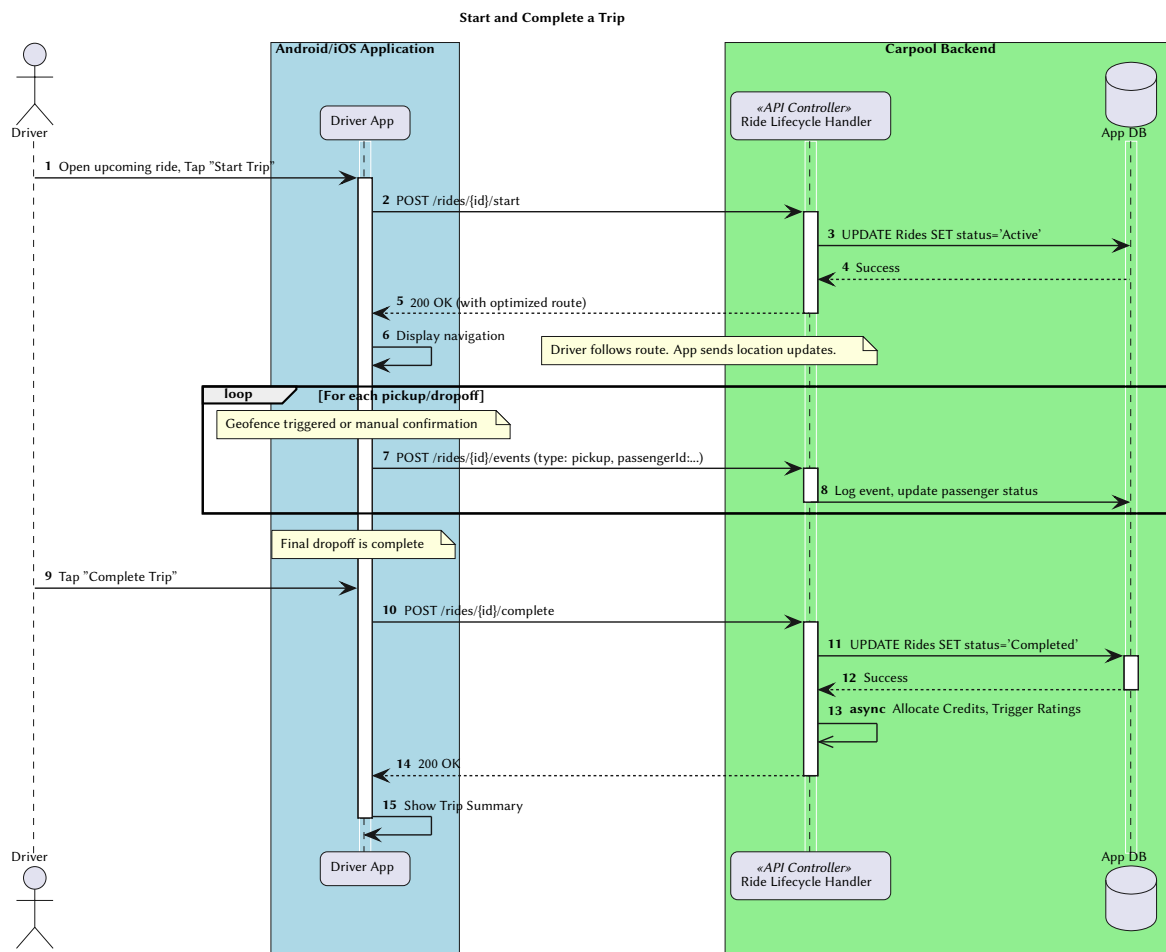


Figure 21: Sequence diagram showing the start and completion of a trip.

11.4.9 Verify Driver Documents

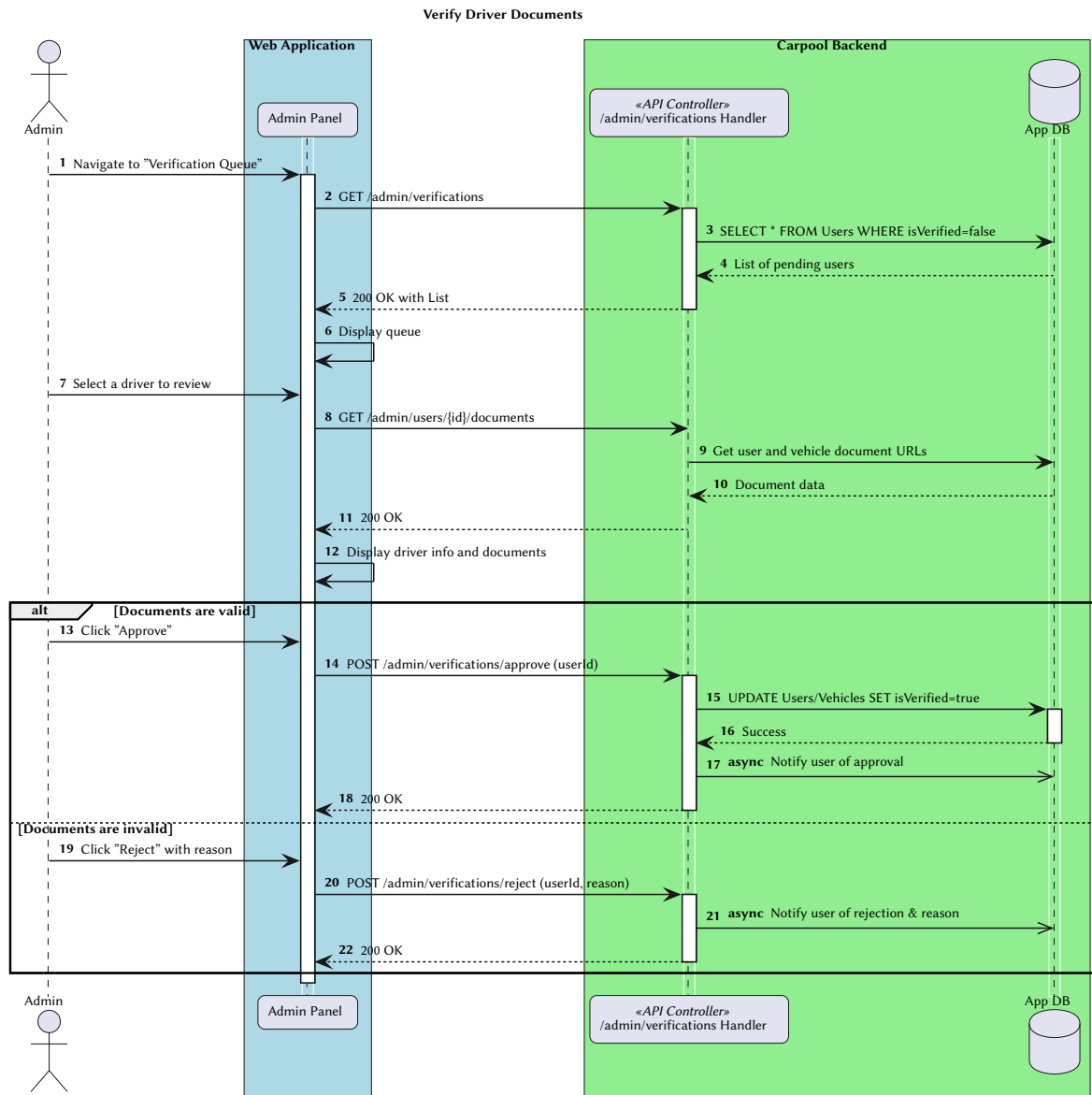


Figure 22: Sequence diagram showing the verification of driver documents.

11.4.10 Suspend a User Account

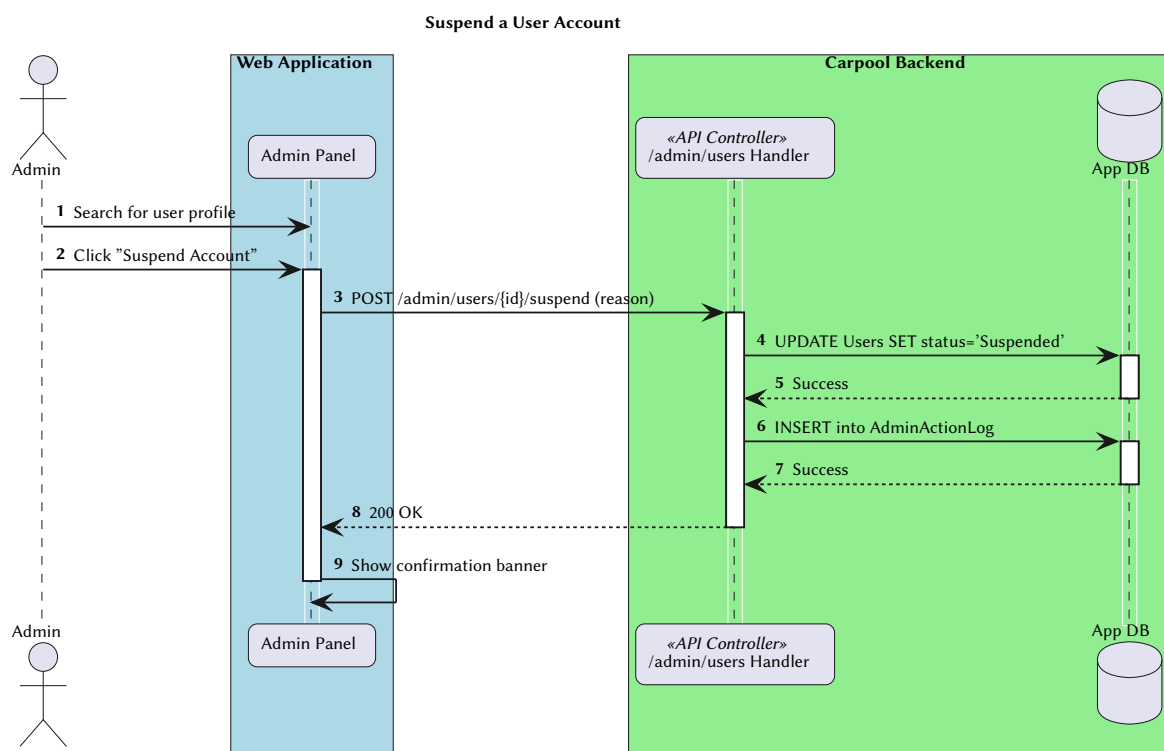


Figure 23: Sequence diagram showing suspension of a user account.

11.4.11 Investigate Ride Dispute

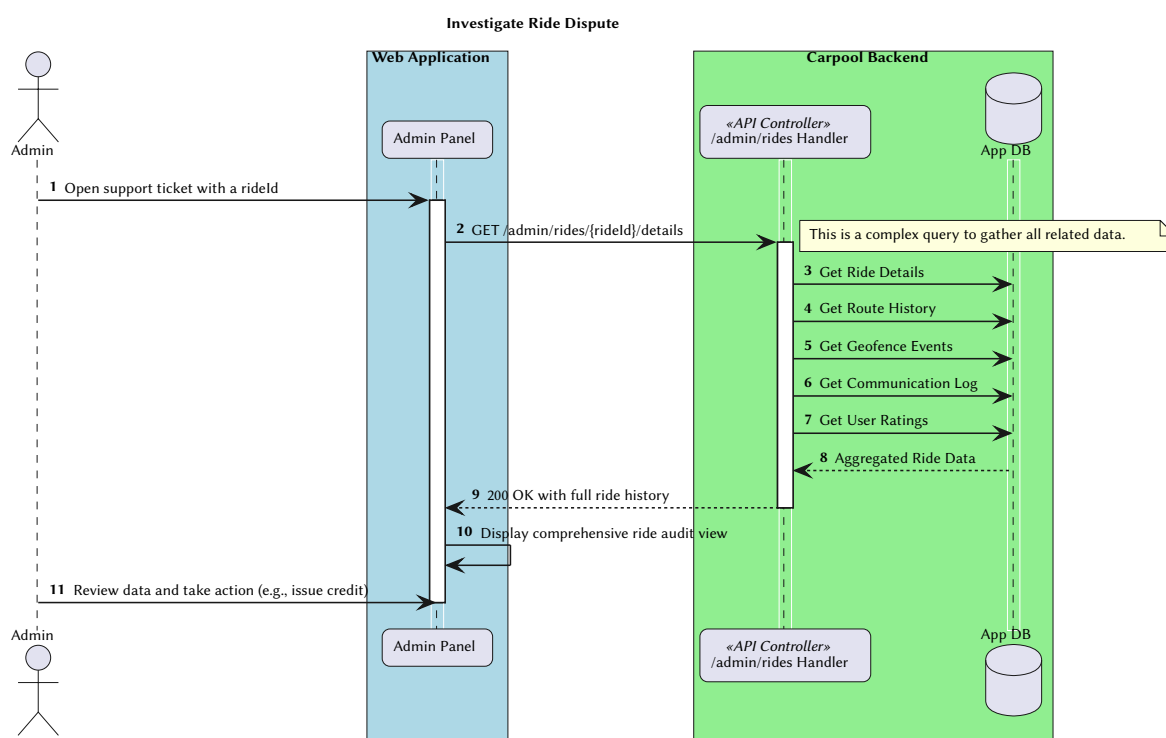


Figure 24: Sequence diagram showing the investigation of ride disputes by Admin.

11.4.12 Respond to SOS Alert

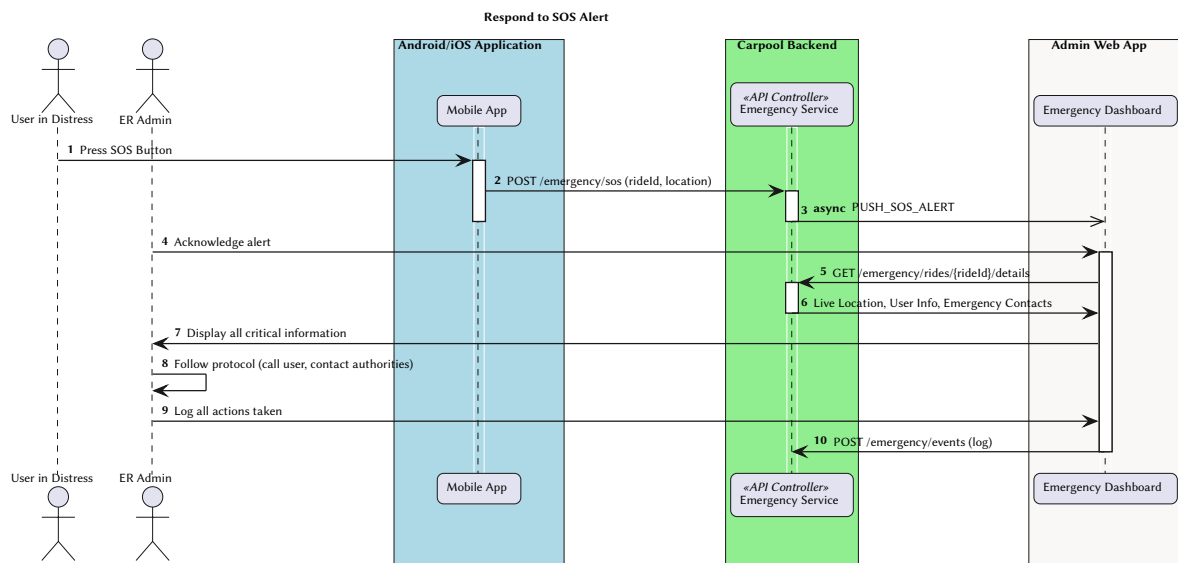


Figure 25: Sequence diagram showing response to SOS alert.

11.4.13 Rate a Completed Ride

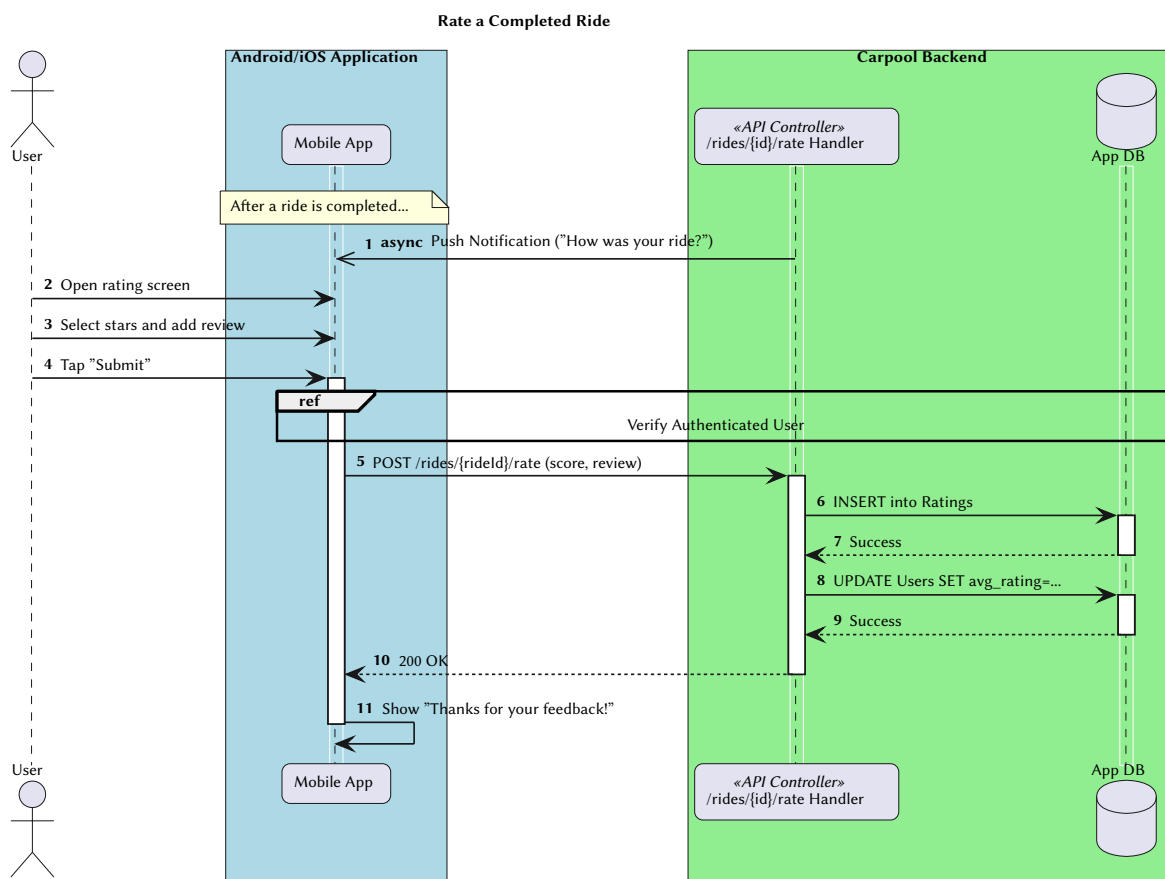


Figure 26: Sequence diagram showing the rating process of a completed ride.

11.4.14 Configure System Parameters

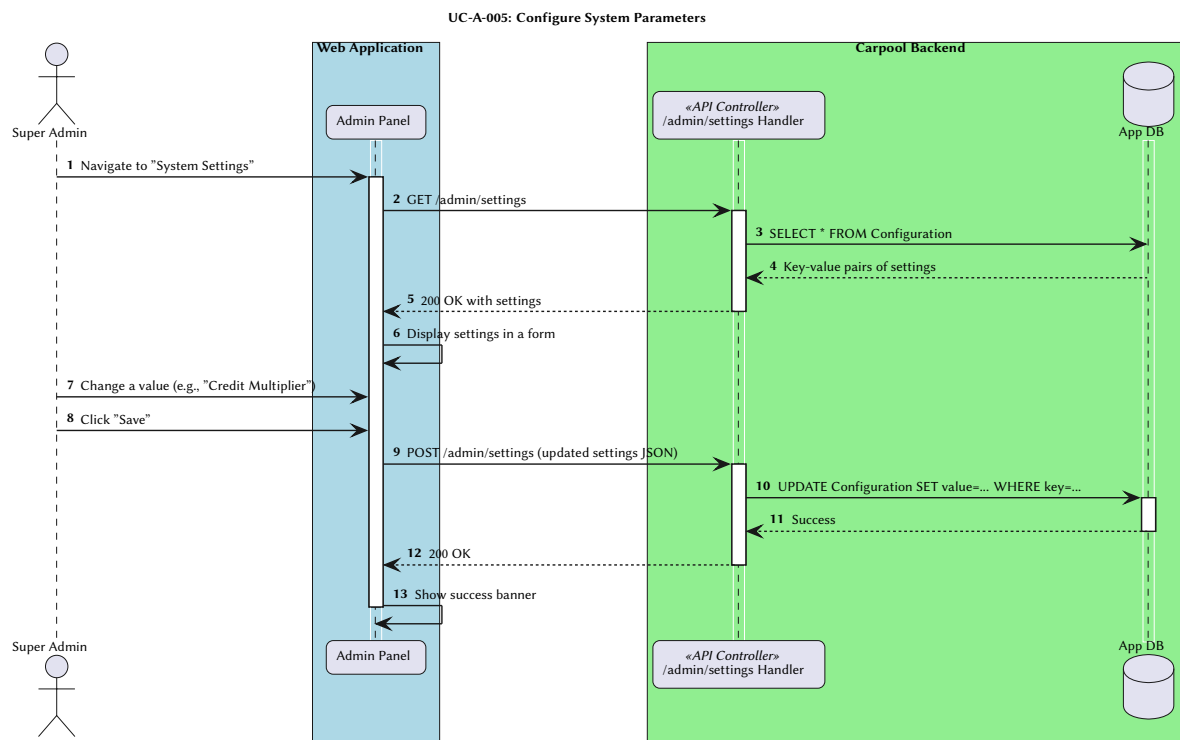


Figure 27: Sequence diagram showing how super-user configures application parameters.

11.4.15 Log Out

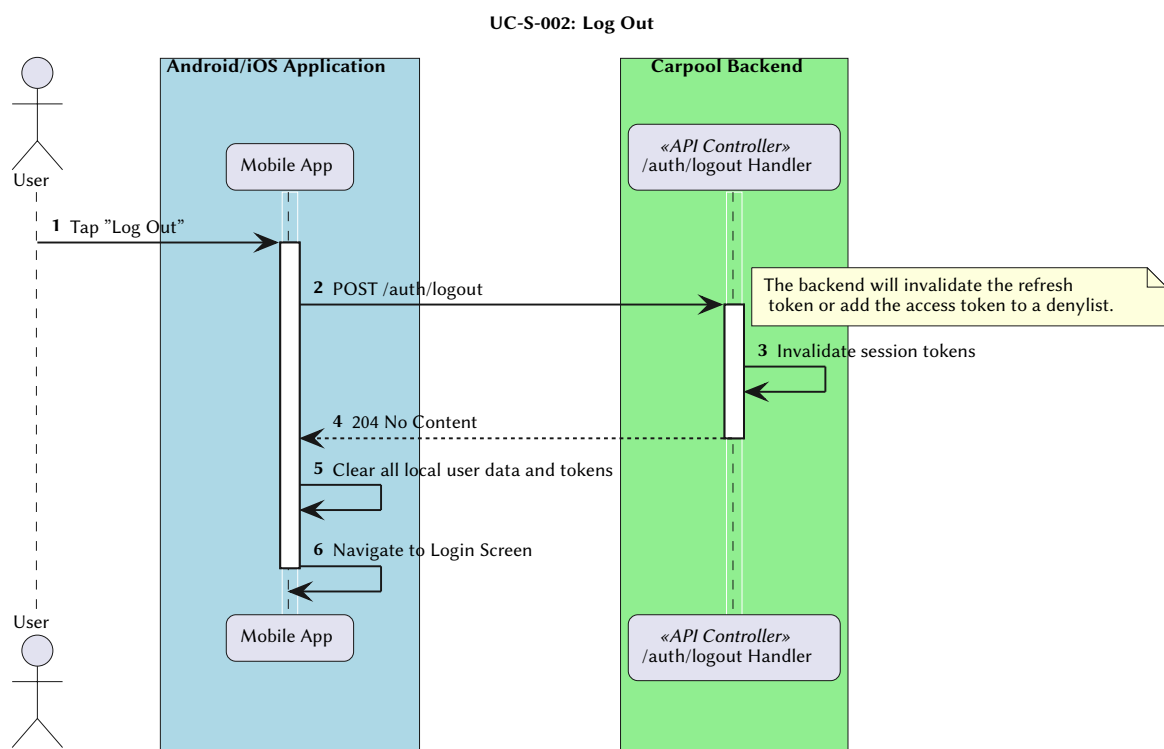


Figure 28: Sequence diagram showing the log our process.

12 User Interface Mock-ups

12.1 Onboarding & Common User Flows

These screens cover the initial user experience from first login to account management.

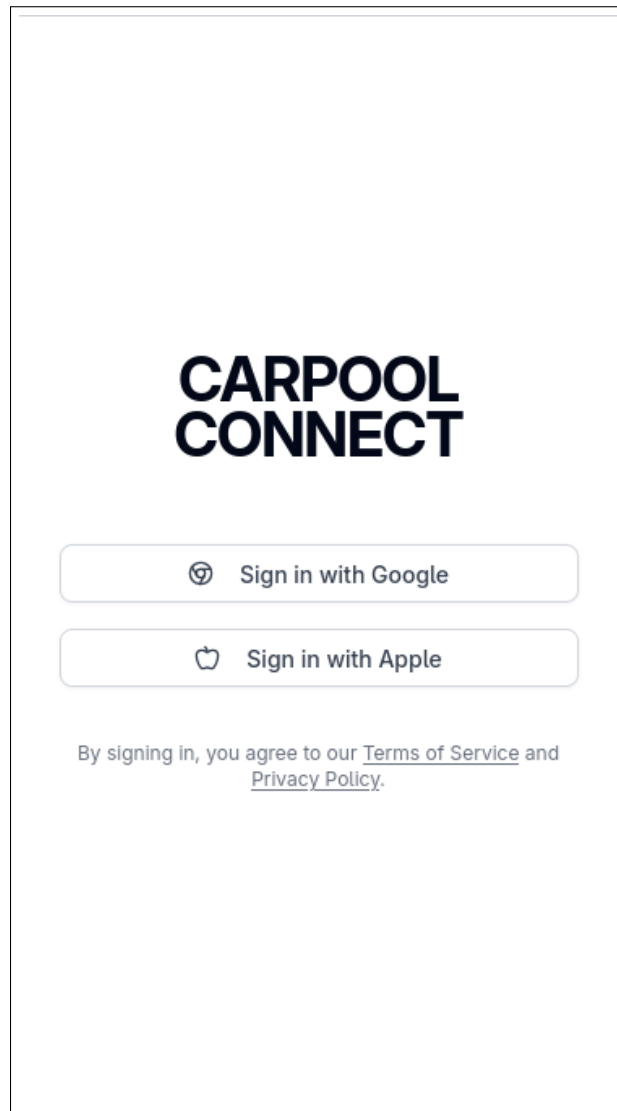


Figure 29: Initial Login (UC-S-001). To provide a fast, password-free entry into the app.

Verify Your Number

A valid phone number is required for safety and communication.

Country:

Ethiopia (+251) ▾

Phone:

e.g., 912345678

Send Code

Figure 30: Phone Number Verification (Post-SSO for New Users). To ensure every user has a verified, unique phone number before they can use any features.

Enter Your Code

We sent a 6-digit code to your number.

5

6

Didn't get a code? [Resend](#)

Verify

Figure 31: OTP Entry (Post-Verification Code Sent). To complete the phone verification loop.

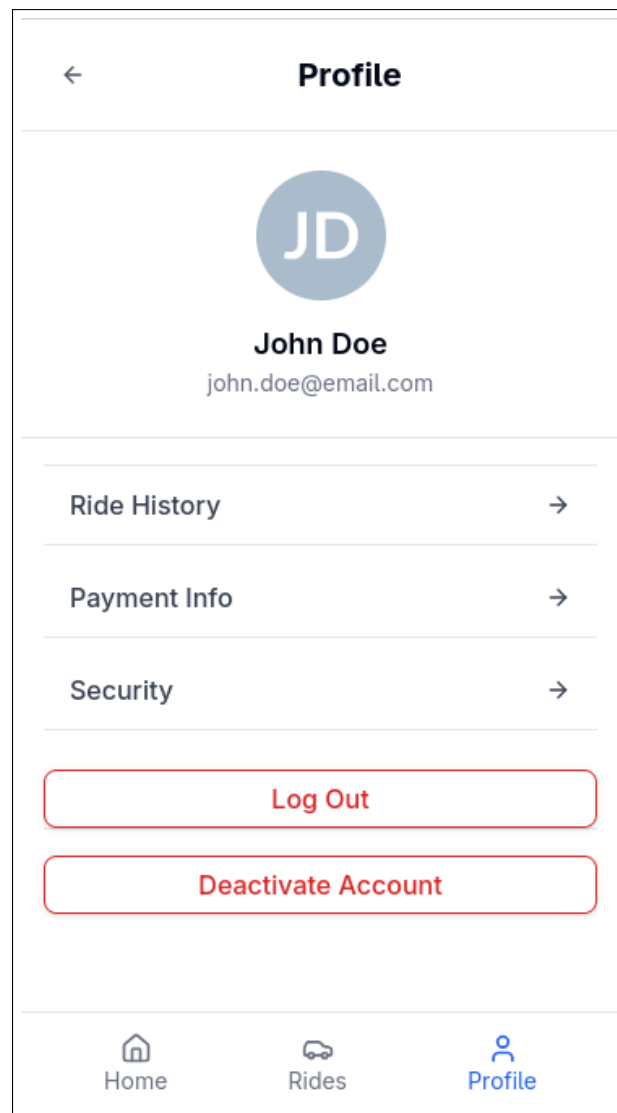


Figure 32: User Profile & Settings (UC-P-001, UC-S-002). To allow users to manage their account details and access critical actions like logout and deactivation.

12.2 Passenger User Flow

This storyboard follows a passenger from requesting a ride to its completion.

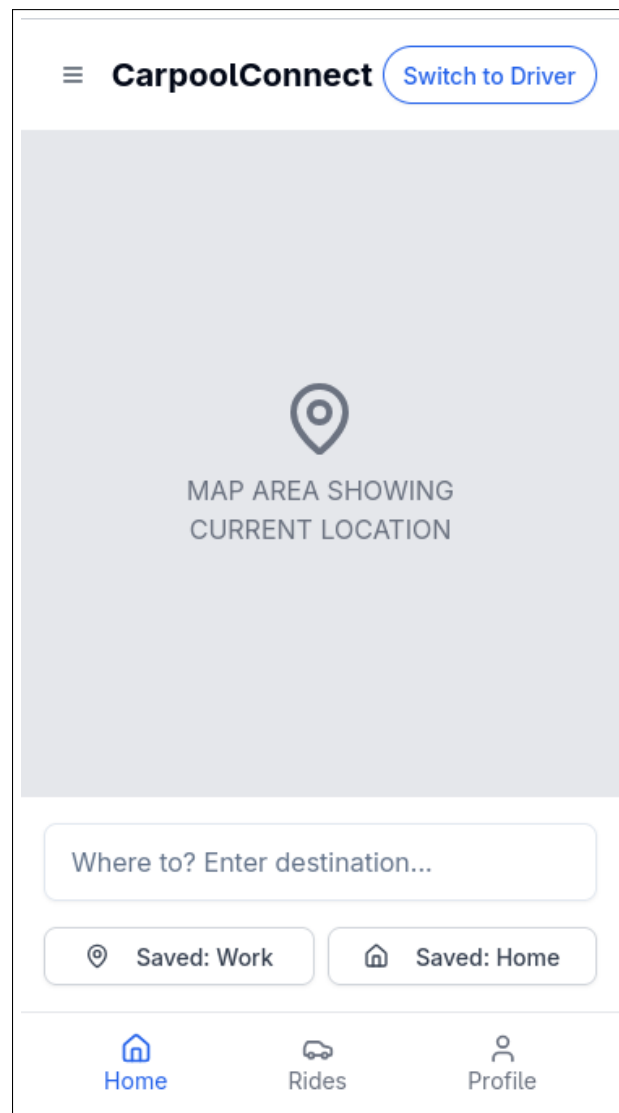


Figure 33: Passenger Home (UC-PA-001). To provide the main entry point for requesting a ride.

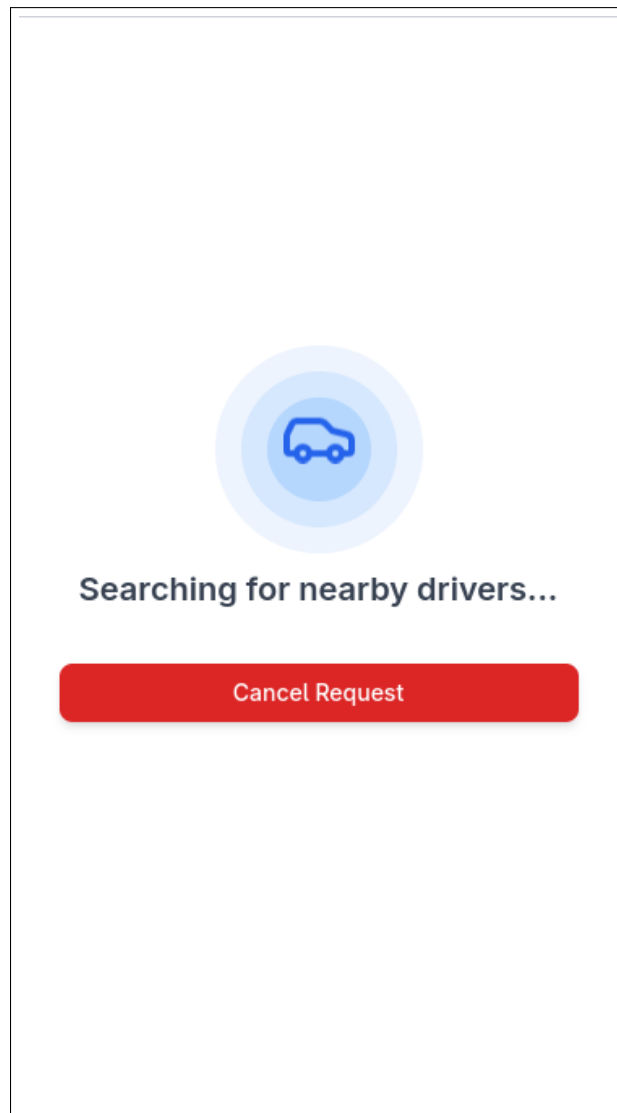


Figure 34: Searching for Ride (Post-Request). To give feedback to the user that the system is actively working on their request.

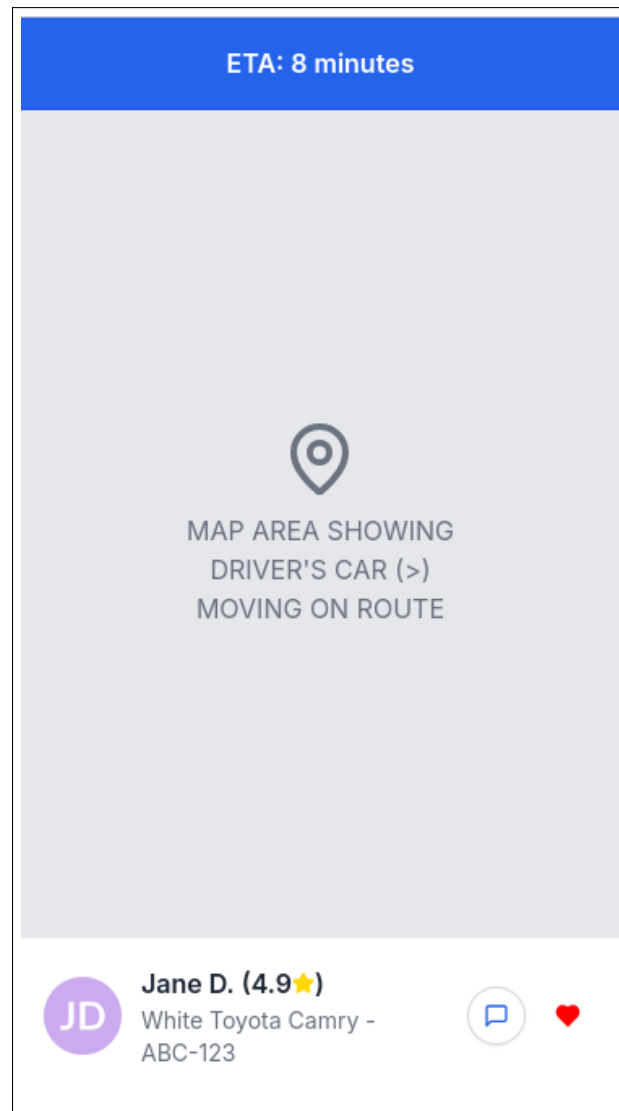
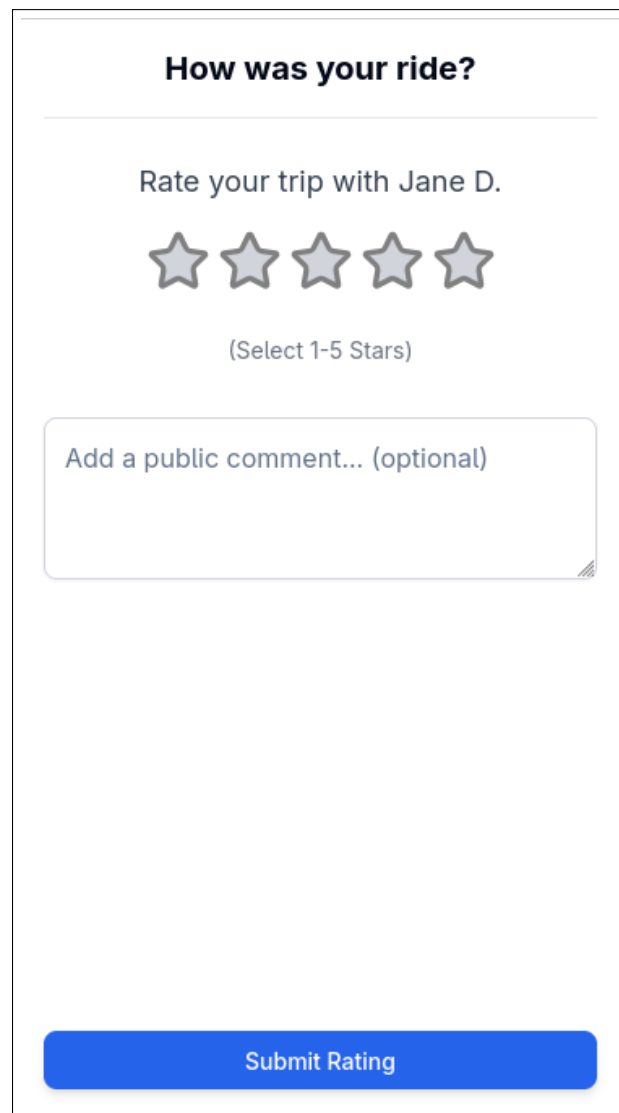


Figure 35: Active Ride - Tracking (UC-PA-002). To provide real-time updates, driver details, and safety features during a ride.



The mockup is a vertical rectangle with a light gray border. At the top, the text "How was your ride?" is centered in a bold, black font. Below this is a horizontal line. The next line of text is "Rate your trip with Jane D." in a standard black font. Underneath is a row of five gray stars. Below the stars is the text "(Select 1-5 Stars)" in a smaller, gray font. Below this is a rounded rectangular text input field with a light gray border and a small pencil icon in the bottom right corner. The text "Add a public comment... (optional)" is inside the field. At the bottom of the form is a solid blue button with the text "Submit Rating" in white.

Figure 36: Ride Rating (UC-G-001). To prompt the user to rate their driver after the trip is completed.

12.3 Driver User Flow

This storyboard follows a driver from offering a ride to its completion.

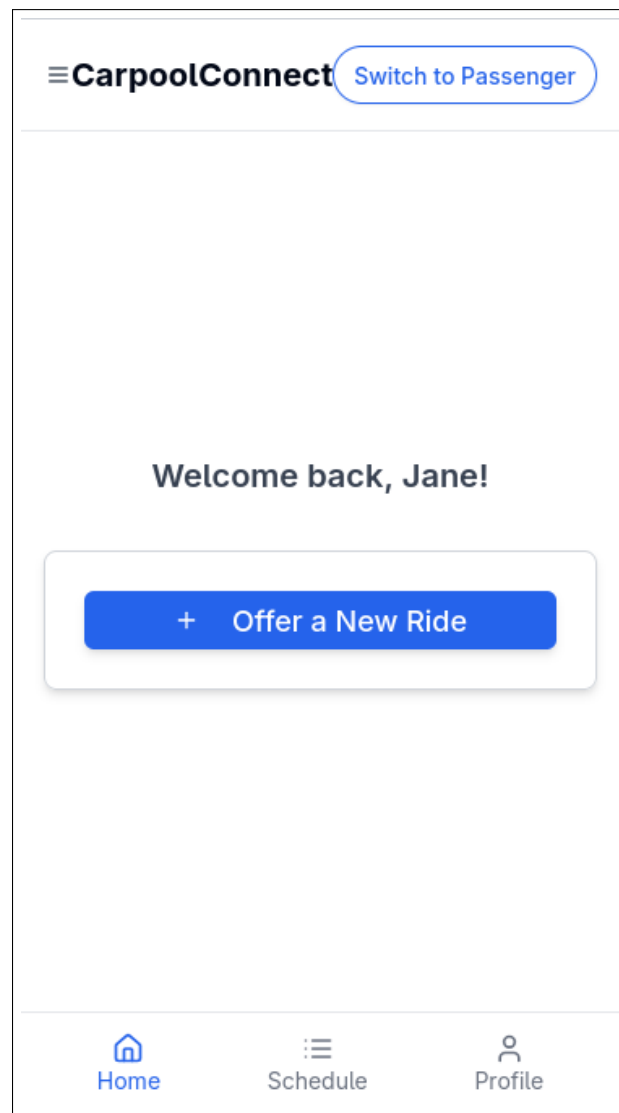
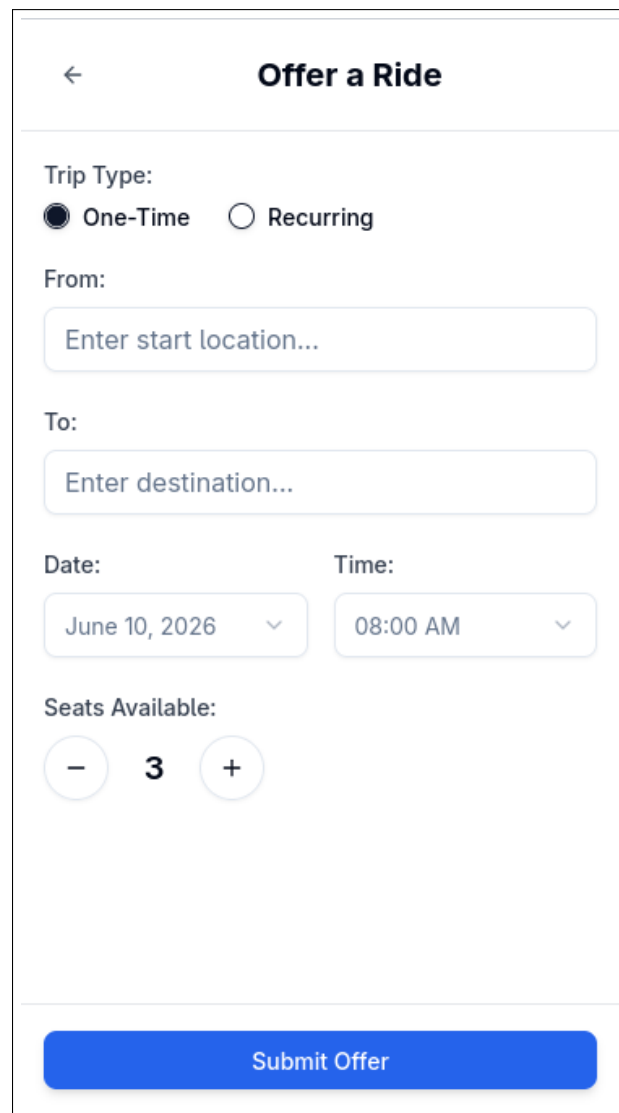


Figure 37: Driver Home - Default. *To provide a clear starting point for a driver to offer a trip.*



The form is titled "Offer a Ride" and includes a back arrow. It contains several input fields and a submit button. The "Trip Type" section has two radio buttons: "One-Time" (selected) and "Recurring". The "From:" field is a text input with the placeholder "Enter start location...". The "To:" field is a text input with the placeholder "Enter destination...". The "Date:" field is a date picker showing "June 10, 2026" with a dropdown arrow. The "Time:" field is a time picker showing "08:00 AM" with a dropdown arrow. The "Seats Available:" section has three circular buttons: a minus sign, the number "3", and a plus sign. At the bottom is a blue "Submit Offer" button.

← **Offer a Ride**

Trip Type:
☒ One-Time ☐ Recurring

From:
Enter start location...

To:
Enter destination...

Date: June 10, 2026 ▼ Time: 08:00 AM ▼

Seats Available:
- 3 +

Submit Offer

Figure 38: Offer Ride Form (UC-D-001/002). To allow the driver to specify all details for a new ride offer.

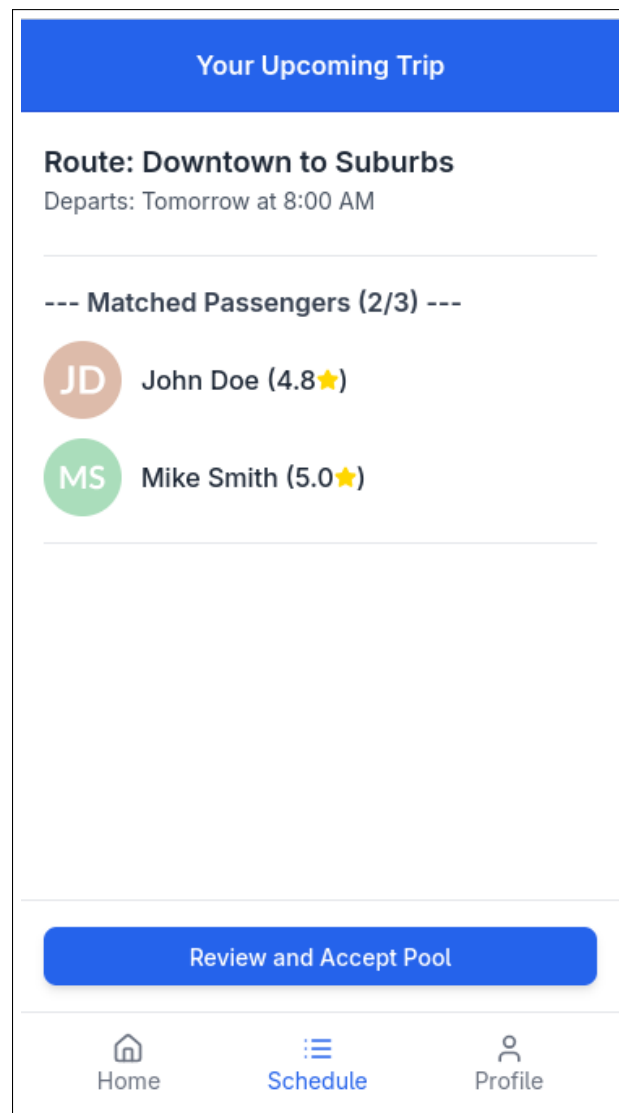


Figure 39: Pending Offer - Matched (UC-D-003). To show the driver their active offer and the passengers matched by the engine.

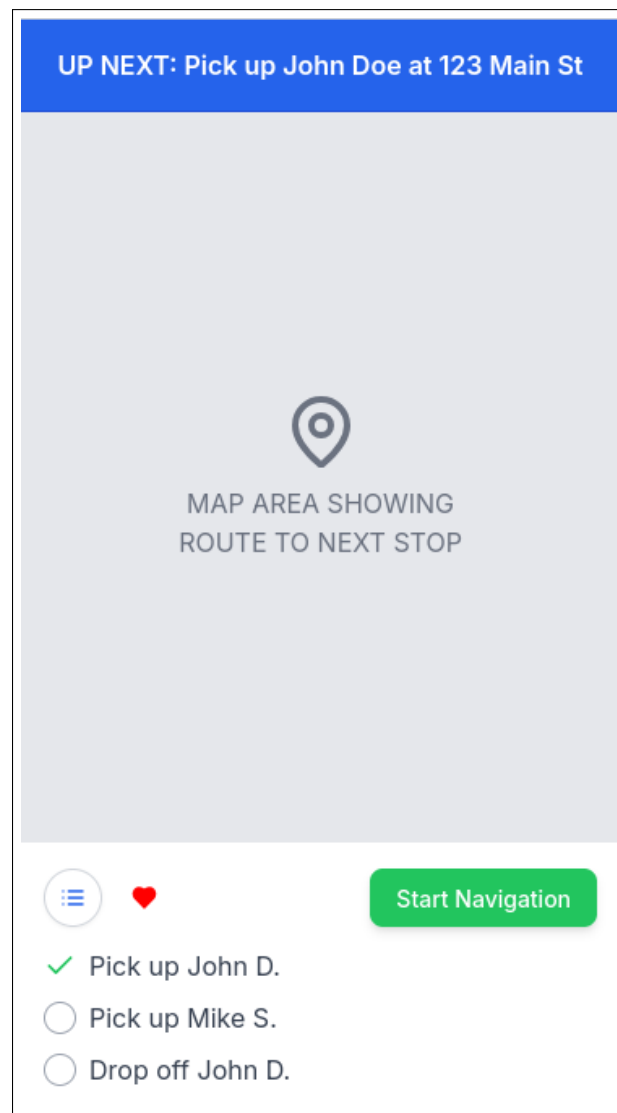


Figure 40: Active Trip - Navigation (UC-D-004). *To provide turn-by-turn guidance for pickups and drop-offs.*

12.4 Admin Panel Flow (Web SPA)

This represents a simplified view of the web-based admin panel.

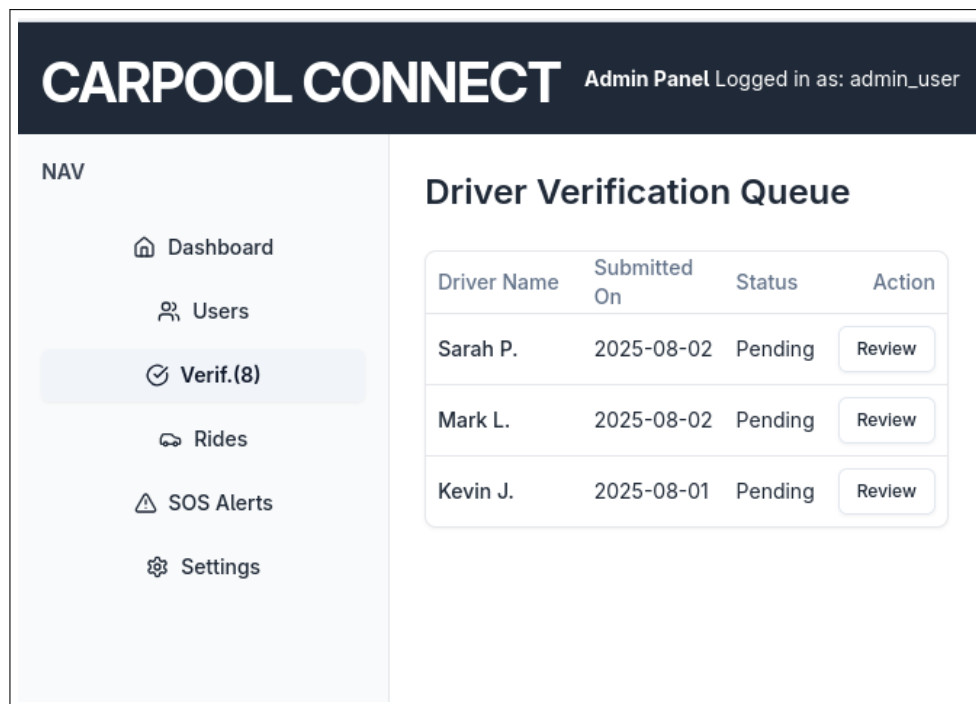


Figure 41: Admin - Verification Queue (UC-A-001). To allow admins to efficiently process pending driver verifications.

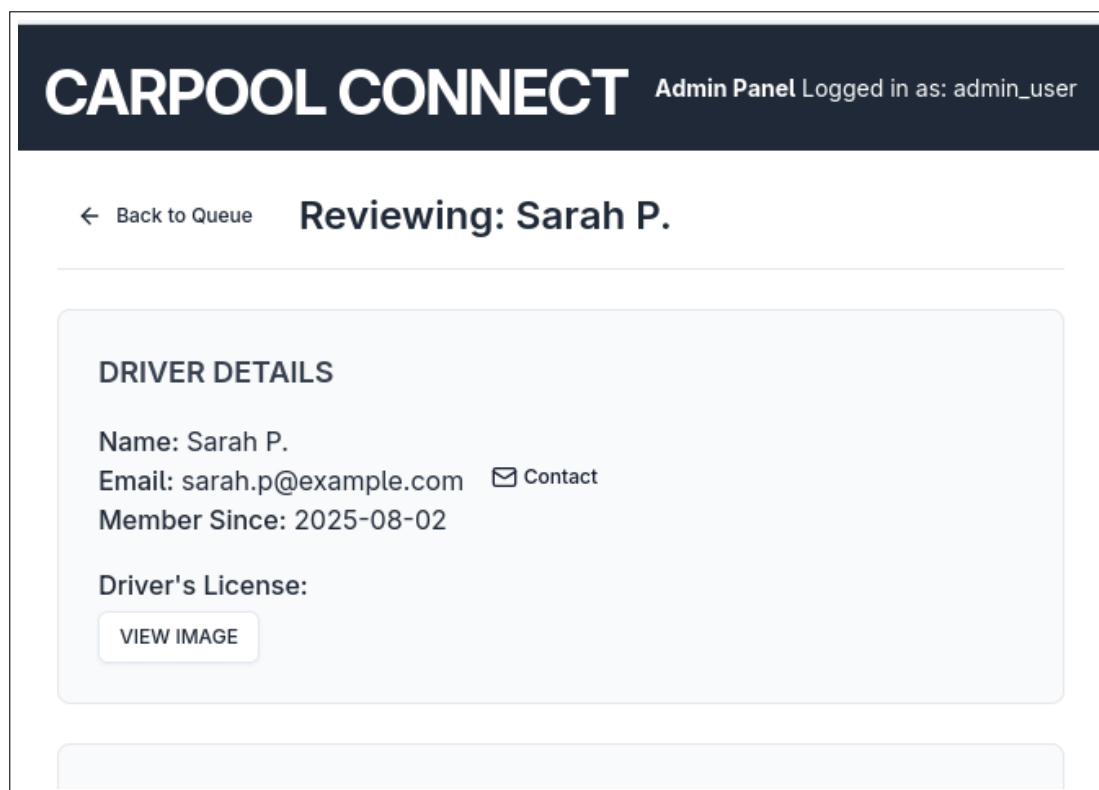


Figure 42: Admin - Verification Detail View. To show all necessary information for an admin to approve or reject a driver.

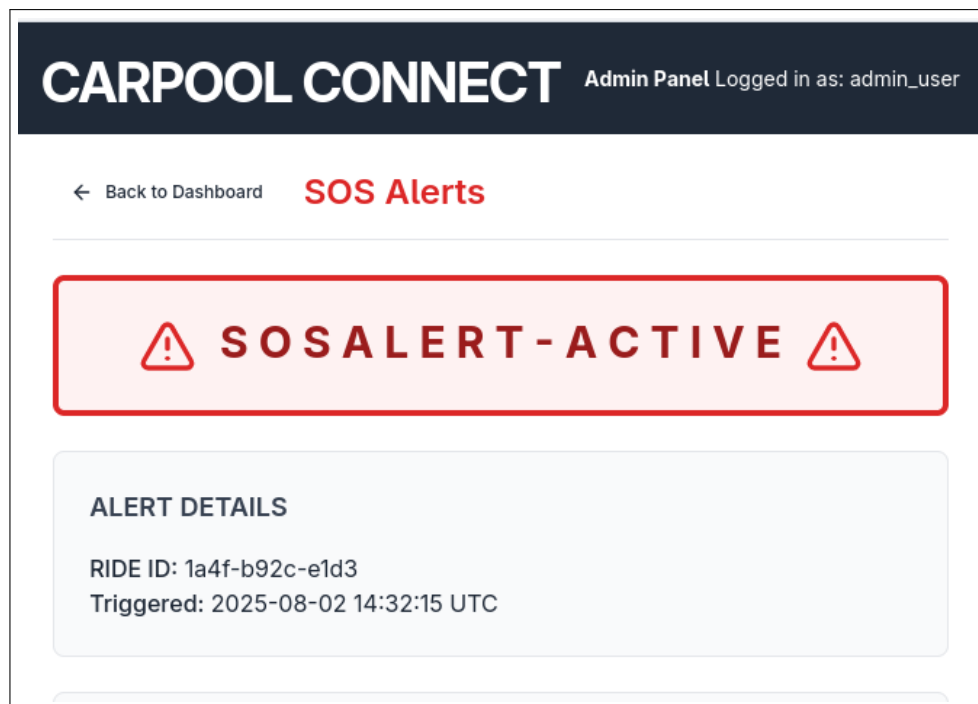


Figure 43: Admin - SOS Alert Dashboard (UC-A-004). *To provide an immediate, high-priority view of a live emergency.*

References

- [1] N. A. a. S. Administration (NASA), *Work Breakdown Structure (Wbs) Handbook*, en. CreateSpace Independent Publishing Platform, May 2018, Google-Books-ID: 65ODtwEACAAJ, ISBN: 978-1-7194-9838-8.